Министерство цифрового развития, связи и массовых коммуникаций РФ
Уральский технический институт связи и информатики (филиал)
ФГБОУ ВО «Сибирский государственный университет
телекоммуникаций и информатики» в г. Екатеринбурге
(УрТИСИ СибГУТИ)

Отчёт
по учебной практике
по МДК 02.02 «Инструментальные средства разработки
программного обеспечения»
на тему «Разработка приложения по варианту ДЭ
и реализация дипломного проекта»

студента IV курса 285 группы

Фамилия Киракосян

Имя, отчество Арсен Артурович

Факультет Инфокоммуникаций, информатики и управления

Специальность 09.02.07 «Информационные системы
и программирование»

Екатеринбург, 2026

Отзыв руководителя

Содержание

Введение 5
1 Демонстрационный экзамен 6
1.1 Анализ предметной области и технического задания 6
1.2 Выбор стека технологий и инструментов разработки 7
1.3 Работа с базой данных 8
1.4 Практическая реализация решения 9
1.5 Интерфейс программы 13
1.6 Тестирование программы 16
1.7 Руководство системному программисту 17
1.7.1 Общие сведения о программе 17
1.7.2 Функции программы 17
1.7.3 Минимальный состав технических средств 17
1.7.4 Минимальный состав программных средств 18
1.7.5 Требования к персоналу (системному программисту) 18
1.8 Структура программы 18
1.8.2 Сведения о составных частях программы 19
1.8.2 Сведения о связях между составными частями программы 20
1.8.3 Сведения о связях с другими программами 20
1.9 Настройка программы 20
1.9.1 Настройка на состав технических средств 20
1.9.2 Настройка на состав программных средств 21
1.10 Проверка программы 21
1.10.1 Описание способов проверки 21
1.11 Сообщения системному программисту 22
2 Дипломный проект 23
2.1 Введение дипломного проекта 23
2.2 Анализ предметной области 23
2.3 Требование к своему программному продукту 24
2.4 Обоснование выбора инструментальных средств разработки 27
2.5 Разработка архитектуры блок-схем программы 28
2.6 Разработка макетов основных форм приложения 29
2.7 Проектирование и разработка базы данных 32
2.8 Разработка пользовательского интерфейса 35
2.9 Доработка приложения 43
2.9.1 Разработка блок-схем функций 43
2.9.2 Разработка функционала 46

Заключение..... 50
Библиография..... 51
Приложение А..... 52
Приложение Б 53

Введение

В современных условиях цифровизации сферы услуг особое значение приобретает автоматизация процессов обслуживания клиентов и внутренней координации сотрудников. В сервисных центрах часто используются устаревшие или разрозненные методы ведения учета, которые не обеспечивают должной скорости обработки обращений и не позволяют эффективно контролировать работу мастеров.

Актуальность темы заключается в необходимости создания надежного программного продукта, который будет учитывать специфику работы предприятия по обслуживанию и ремонту климатического оборудования (кондиционеров, систем вентиляции и отопления). Автоматизация позволит повысить эффективность работы всего отдела, существенно сократить время реакции на заявки, упростить контроль за выполнением ремонта на всех этапах, а также улучшить качество обслуживания клиентов и планирование ресурсов.

Целью учебной практики является разработка и последующая модификация программного модуля для учета заявок на ремонт климатического оборудования в рамках сдачи демонстрационного экзамена.

Для достижения данной цели необходимо выполнить следующие задачи:

- проанализировать техническое задание и специфику предметной области сервисного центра для выявления основных требований к системе;
- спроектировать и реализовать логичную базу данных в 3-й нормальной форме для надежного хранения информации о заказчиках, заявках и сотрудниках;
- разработать интерфейс и программный модуль для регистрации заявок, изменения их статусов и назначения ответственных исполнителей;
- модифицировать систему, внедрив функционал для новой роли «Менеджер по качеству», чтобы оперативно решать проблемы с задержками ремонта;
- оформить проектную документацию, включая руководство системного программиста, в соответствии с установленными стандартами.

1 Демонстрационный экзамен

1.1 Анализ предметной области и технического задания

В рамках первого этапа выполнения практической работы был проведен подробный анализ предметной области и требований заказчика. Предметная область охватывает процессы обслуживания и ремонта климатического оборудования, к которому относятся кондиционеры, а также вентиляционные и отопительные системы. Главной целью разработки программного модуля является автоматизация обработки заявок от клиентов, что позволит существенно облегчить работу профильных специалистов, сократить время реакции на обращения и более эффективно планировать рабочие ресурсы сервисного центра.

Анализ бизнес-процессов показал, что жизненный цикл заявки состоит из нескольких ключевых этапов:

- создание заявки: клиент предоставляет информацию о неисправности, указывает тип и модель оборудования, описывает проблему и оставляет свои контактные данные (ФИО, номер телефона);
- регистрация: оператор сервисной службы принимает обращение и присваивает ему уникальный идентификатор в системе;
- обработка: на данном этапе происходит анализ проблемы, определяется приоритетность задачи и назначается конкретный специалист-исполнитель;
- исполнение: мастер осуществляет фактический ремонт оборудования, при необходимости заказывает нужные комплектующие или координирует свои действия с другими сотрудниками;
- отчетность и мониторинг: по завершении работ специалист фиксирует затраченные ресурсы (время, запчасти), а система позволяет анализировать общие показатели качества и скорости обслуживания.

Согласно техническому заданию, разработка ведется для заказчика ООО «Конди», исполнителем выступает сервисный центр «IT-Сфера». Программный продукт должен обеспечивать выполнение следующих основных функций:

- добавление новых заявок и их последующее редактирование (изменение этапа выполнения, описания проблемы или ответственного лица);
- отслеживание текущих статусов обращений, поиск по номеру или параметрам, а также получение уведомлений;
- ведение статистики работы отдела (подсчет количества выполненных заявок, расчет среднего времени ремонта, вывод статистики по типам неисправностей);
- разграничение прав доступа пользователей с помощью авторизации (логин и пароль).

Программа должна быть кроссплатформенной (с обязательной поддержкой ОС семейства Windows), иметь интуитивно понятный интерфейс и обеспечивать быстрый отклик при работе с базой данных.

1.2 Выбор стека технологий и инструментов разработки

В соответствии с техническим заданием, выбор языка программирования и системы управления базами данных (СУБД) для создания программного модуля остается на усмотрение разработчика. Для реализации полнофункциональной системы учета заявок на ремонт климатического оборудования был выбран современный веб-ориентированный стек технологий: C#, ASP.NET Core, HTML, JavaScript и PostgreSQL.

Данный выбор обоснован необходимостью создания надежного, безопасного и масштабируемого приложения:

C# и ASP.NET Core (Backend): использование серверной платформы ASP.NET Core на базе языка программирования C# обеспечивает высокую производительность и гибкость разработки. Этот фреймворк позволяет эффективно реализовать сложную бизнес-логику системы. ASP.NET Core является кроссплатформенным решением, что гарантирует стабильную работу программного обеспечения на операционных системах семейства Windows. Встроенные механизмы аутентификации платформы позволяют надежно реализовать авторизацию по логину и паролю, а также разграничить доступ к данным в зависимости от роли пользователя (оператор, специалист, менеджер по качеству).

HTML и JavaScript (Frontend): для разработки клиентской части приложения применялась связка языка гипертекстовой разметки HTML и языка программирования JavaScript. HTML формирует структуру веб-страниц, таблиц и форм для ввода данных по заявкам. JavaScript, в свою очередь, обеспечивает интерактивность простого и интуитивно понятного пользовательского интерфейса. Кроме того, использование JS необходимо для реализации функционала генерации QR-кодов на стороне клиента, что требуется для сбора отзывов о качестве работы сервисной службы.

СУБД PostgreSQL и pgAdmin 4: в качестве базы данных была выбрана надежная объектно-реляционная СУБД PostgreSQL. Она обладает всем необходимым инструментарием для проектирования схемы данных в 3-й нормальной форме (3НФ) с обеспечением строгой ссылочной целостности. Для администрирования сервера баз данных, настройки первичных и внешних ключей, а также импорта начальных данных из предоставленных файлов использовалась графическая среда управления pgAdmin 4.

Microsoft Visual Studio 2026 (IDE): основной интегрированной средой разработки выступила Visual Studio 2026. Данная IDE предоставляет мощные встроенные инструменты для написания чистого кода, его форматирования и проведения функционального тестирования. Встроенный отладчик позволил оперативно выявлять ошибки, настраивать программную обработку исключительных ситуаций и реализовывать информативные уведомления о действиях пользователя.

1.3 Работа с базой данных

Для надежного хранения информации была спроектирована реляционная база данных под управлением СУБД PostgreSQL. В соответствии с заданием, главным требованием к разработке ER-модели стало приведение структуры к третьей нормальной форме (3НФ) и обеспечение строгой ссылочной целостности.

На основе анализа технического задания были выделены следующие сущности и их атрибуты:

- roles (роли): справочник системных ролей (roleID – первичный ключ (PK); roleName).
- statuses (статусы): справочник этапов выполнения работ (statusID – PK; statusName).
- users (пользователи): единая таблица для хранения данных всех участников системы, включая клиентов, мастеров и менеджеров (userID – PK; fio; phone; login; password; roleID – внешний ключ (FK)).
- requests (заявки): центральная таблица учета обращений (requestID – PK; startDate; climateTechType; climateTechModel; problemDescription; statusID – FK; completionDate; repairParts; masterID – FK; clientID – FK).
- comments (комментарии): таблица для фиксации дополнительных сведений и информации о заказанных комплектующих (commentID – PK; message; masterID – FK; requestID – FK).

Для обеспечения целостности данных связи между сущностями организованы по типу «один ко многим» (1:N) с использованием внешних ключей:

- роли и пользователи: одна роль может быть назначена нескольким пользователям;
- статусы и заявки: один статус может быть присвоен множеству заявок;
- пользователи (мастера) и заявки: один мастер может выполнять множество заявок;
- пользователи (клиенты) и заявки: один клиент может оставить множество заявок на ремонт;
- заявки и комментарии: одна заявка может иметь несколько комментариев от специалиста.

Разработанная логическая структура исключает дублирование данных, предотвращает аномалии обновления и обеспечивает высокую скорость выполнения запросов при формировании статистической отчетности.

ERD-диаграмма системы представлена на рисунке 1.1.

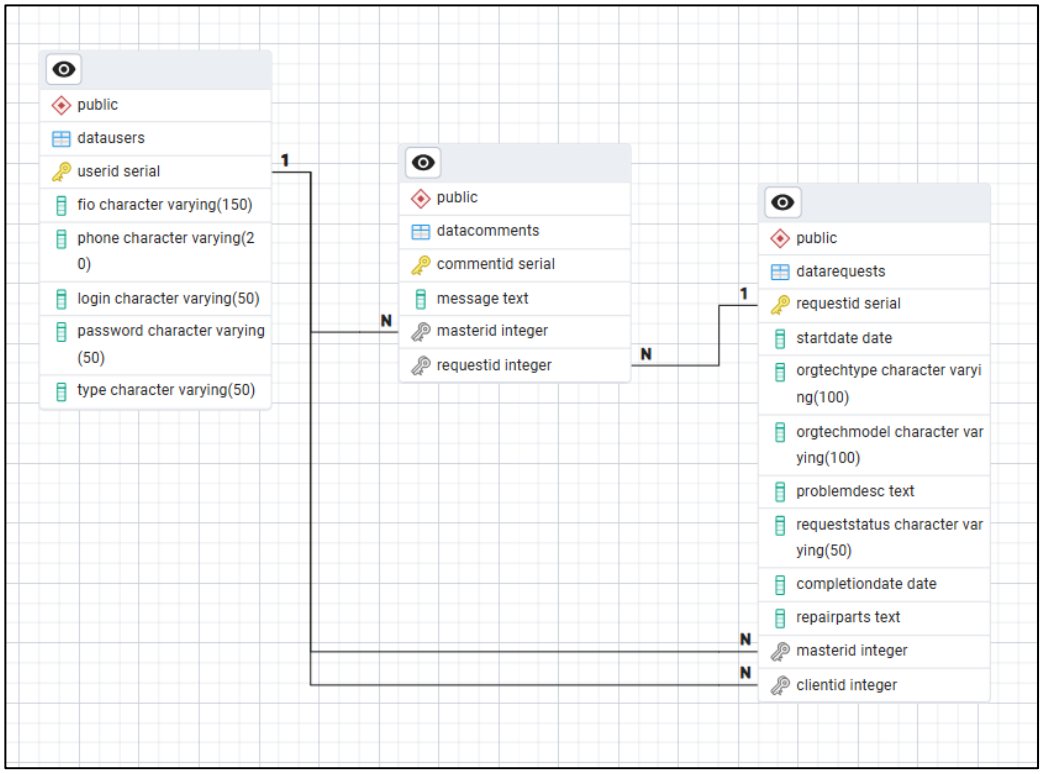

Рисунок 1.1 – ERD-диаграмма

Запрос создание базы данных представлен в приложении А.

1.4 Практическая реализация решения

В соответствии с требованиями технического задания, перед непосредственным написанием программного кода был проведен этап проектирования логики работы приложения. Для наглядной визуализации процессов обработки данных были разработаны графические алгоритмы (блок-схемы), выполненные в строгом соответствии с правилами оформления по ГОСТ 19.701.

В рамках проекта были спроектированы две основные блок-схемы:

Основной алгоритм учета заявок: Первая блок-схема описывает базовый алгоритм работы системы учета заявок на ремонт климатического оборудования. Она наглядно демонстрирует общий жизненный цикл обращения: получение входных данных от клиента при регистрации, проверку условий, изменение статусов заявки и формирование выходных данных при завершении ремонта (Рисунок 1.2)

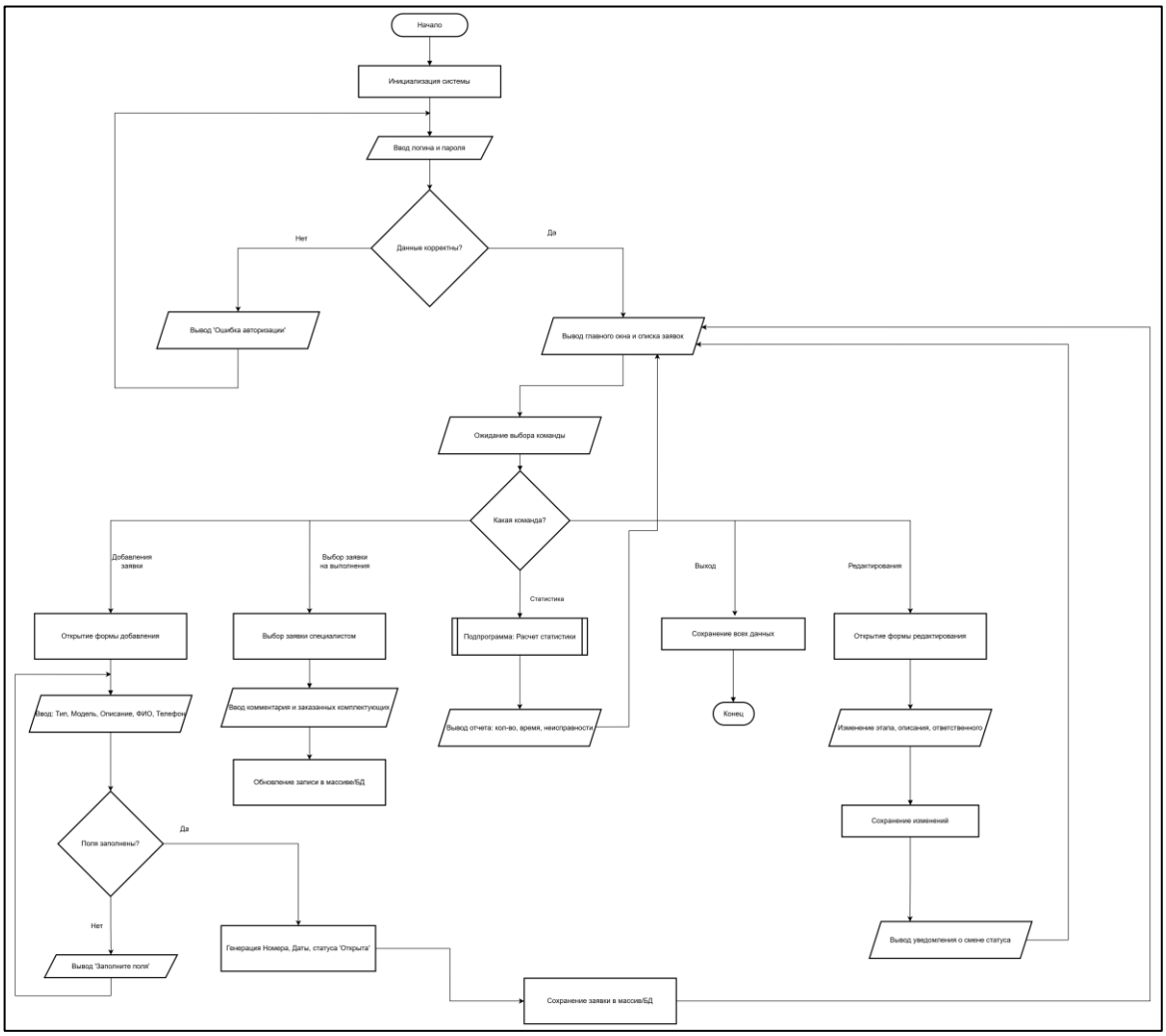

Рисунок 1.2 – Основная блок-схема

Детализированный алгоритм функции расчета: Вторая блок-схема выполняет глубокую детализацию одной из ключевых вычислительных функций модуля – расчета статистики работы отдела обслуживания. Данный алгоритм описывает последовательность циклов и условий, необходимых для подсчета количества выполненных заявок, вычисления среднего времени ремонта и группировки данных для статистики по типам неисправностей (Рисунок 1.3)

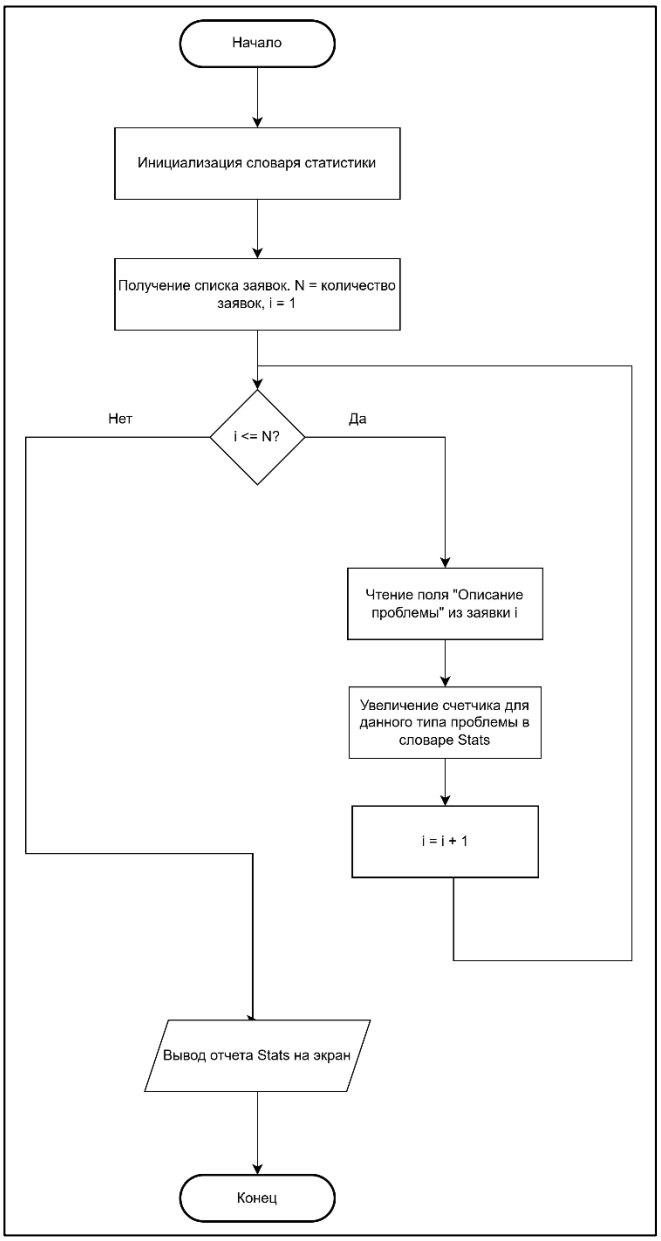

Рисунок 1.3 – Детализированная блок-схема

Программное обеспечение реализовано в виде веб-приложения на платформе ASP.NET Core с использованием архитектурного шаблона MVC (Model-View-Controller), что позволяет логически разделить ответственность между компонентами системы.

Основные компоненты системы:

- Model (Модели): Сущности базы данных реализованы классами User, Role, Request, Status, Comment. Для привязки классов к таблицам PostgreSQL применены атрибуты [Table("...")]. Между моделями строго прописаны навигационные свойства, отражающие связи: заявка связана со статусом, заказчиком и назначенным специалистом; комментарий привязан к конкретной заявке и специалисту.

- Data (Слой данных): Для работы с базой данных используется контекст ApplicationDbContext. Он содержит наборы сущностей (DbSet) и конфигурирует

связи через метод OnModelCreating. В соответствии с требованиями к целостности данных, для предотвращения появления «осиротевших» записей установлено ограничение каскадного удаления DeleteBehavior.Restrict (невозможно удалить пользователя или статус, если в системе существуют привязанные к ним заявки).

- View (Представления): Пользовательский интерфейс реализован с помощью Razor-представлений (HTML/CSS/JS). Данные передаются на страницу через строгую типизацию (@model) и объект ViewBag (например, для формирования выпадающих списков при выборе типа оборудования, статуса или мастера). На стороне клиента (JS) также реализован скрипт генерации QR-кода для оценки работы сервисной службы.

- Controller (Контроллеры): Логика обработки HTTP-запросов распределена по контроллерам: RequestsController (основная работа с заявками), AccountController (аутентификация и сессии), CommentsController (работа с комментариями и запчастями) и StatisticsController (расчет и вывод аналитики).

- Разделение прав доступа и аутентификация: Вход в систему реализован в AccountController. При успешной проверке логина и пароля формируется набор утверждений (Claims: ФИО, роль, ID), который сохраняется в защищенной cookie-сессии. Доступ к контроллерам ограничивается атрибутом [Authorize], а права детализируются внутри методов:

- Заказчик: может создавать заявки и изменять описание проблемы в открытых обращениях.

- Специалист: может изменять этап выполнения (статус), детали ремонта и добавлять комментарии.

- Оператор: управляет всем циклом регистрации и имеет права на назначение специалистов.

- Менеджер по качеству: наделен расширенными правами для привлечения мастеров к сложным ремонтам и продления сроков выполнения заявки.

Работа с заявками (RequestsController):

- Список заявок: формируется запрос к контексту БД с подгрузкой связанных таблиц (.Include()). Реализованы фильтры по номеру, статусу и поиск по параметрам.

- Карточка заявки: загружает полную информацию по ID, включая тип климатического оборудования, модель и описание проблемы. Подгружается список комментариев; кнопка добавления комментария доступна только назначенному на ремонт мастеру.

- Создание заявки: загружаются необходимые справочники. При сохранении дата фиксируется автоматически, устанавливается статус «Открыта заявка», а введенные данные (телефон, ФИО) проходят валидацию.

- Редактирование: набор доступных для изменения полей динамически меняется в зависимости от роли текущего пользователя (от изменения описания проблемы заказчиком до смены ответственного менеджером).

- Удаление: доступно только административному персоналу.

Модули комментариев и аналитики:

- CommentsController обеспечивает добавление текстовых записей к заявкам. Специалист использует этот функционал для фиксации информации о проделанной работе и заказанных комплектующих.

- StatisticsController отвечает за расчет показателей отдела обслуживания. Реализованы LINQ-запросы для группировки данных: подсчет количества выполненных заявок, вычисление среднего времени выполнения и формирование статистики по типам неисправностей.

- Обработка исключительных ситуаций и интерфейс: Для повышения удобства использования реализована программная обработка ошибок:

- Использование встроенных ответов NotFound, Forbid для корректной обработки отсутствующих данных и нарушений прав доступа.

- Глобальный перехват исключений при сохранении в БД (DbUpdateException) с выводом понятного сообщения пользователю.

Вывод информативных уведомлений через TempData (уведомления об успешном создании заявки, смене статуса или предупреждения о неотвратимых операциях, таких как удаление). Окна сообщений содержат соответствующие заголовки и инструкции для пользователя.

1.5 Интерфейс программы

Пользовательский интерфейс веб-приложения спроектирован интуитивно понятным и простым, что обеспечивает высокий уровень удобства для всех групп пользователей. Все формы, таблицы и компоненты системы выполнены в едином, согласованном визуальном стиле с обязательным наличием заголовков на каждом окне. Для предотвращения неверных действий внедрена система информативных визуальных подсказок и уведомлений, выводящая сообщения с пиктограммами об ошибках или успешных операциях. Кроме того, для работы с разными сущностями используются раздельные логические формы, обеспечивающие удобную и последовательную навигацию по приложению.

Интерфейс программы представлен на рисунках 1.4-1.10.

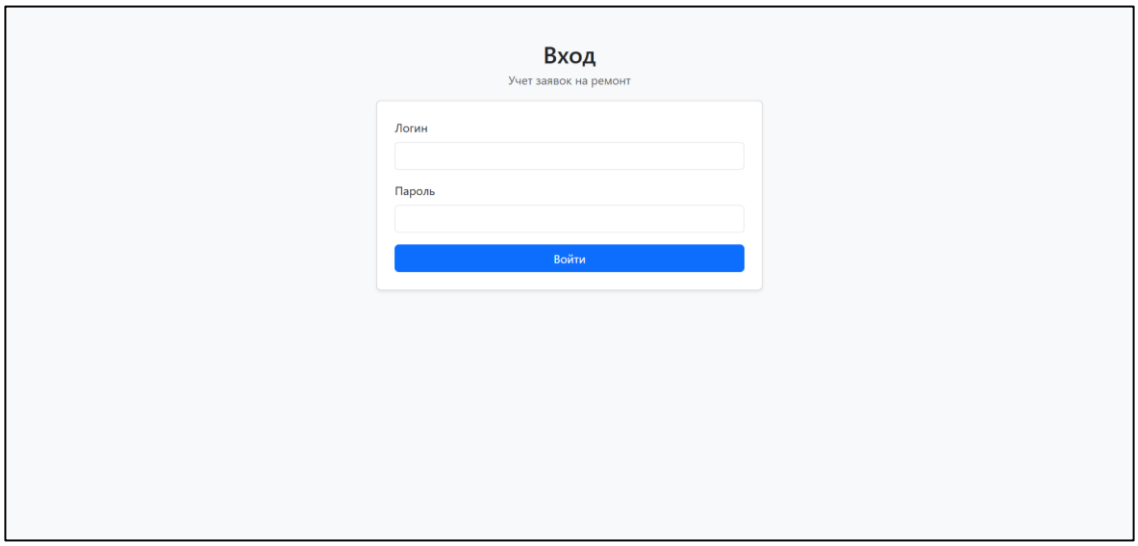

Рисунок 1.4 – Авторизация

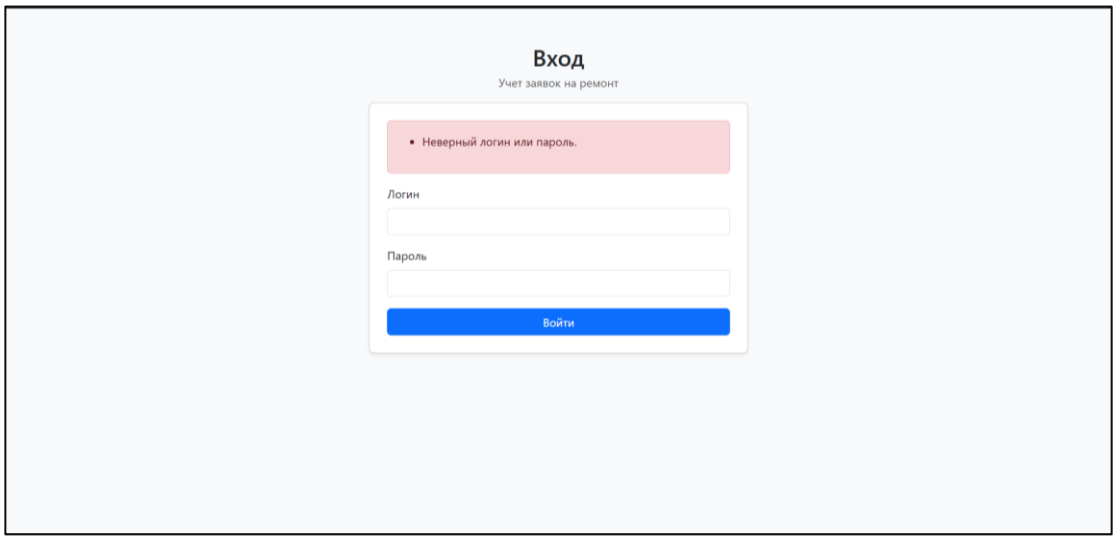

Рисунок 1.5 – Ошибка при введенных неверных данных

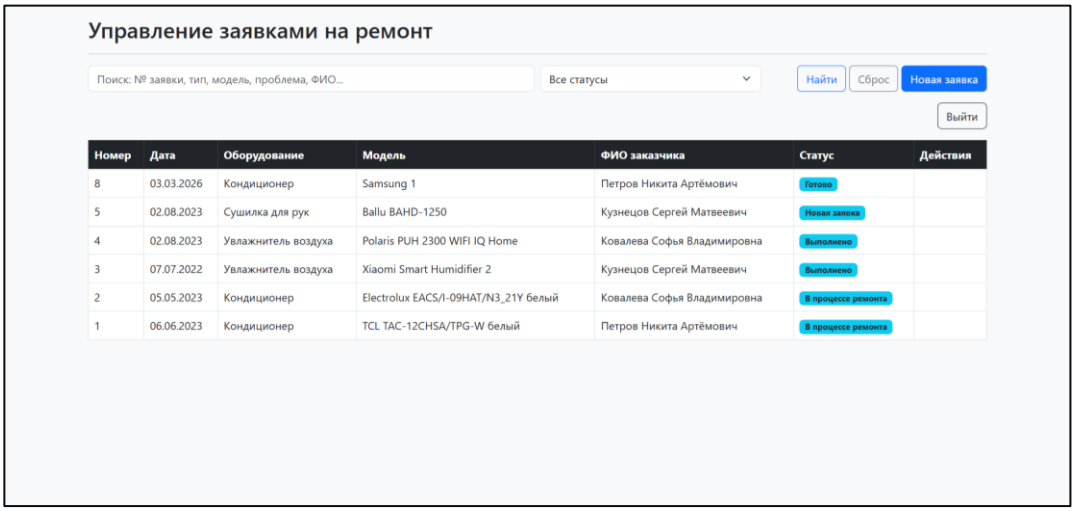

Рисунок 1.6 – Главная страница

Рисунок 1.7 – Добавление новой заявки

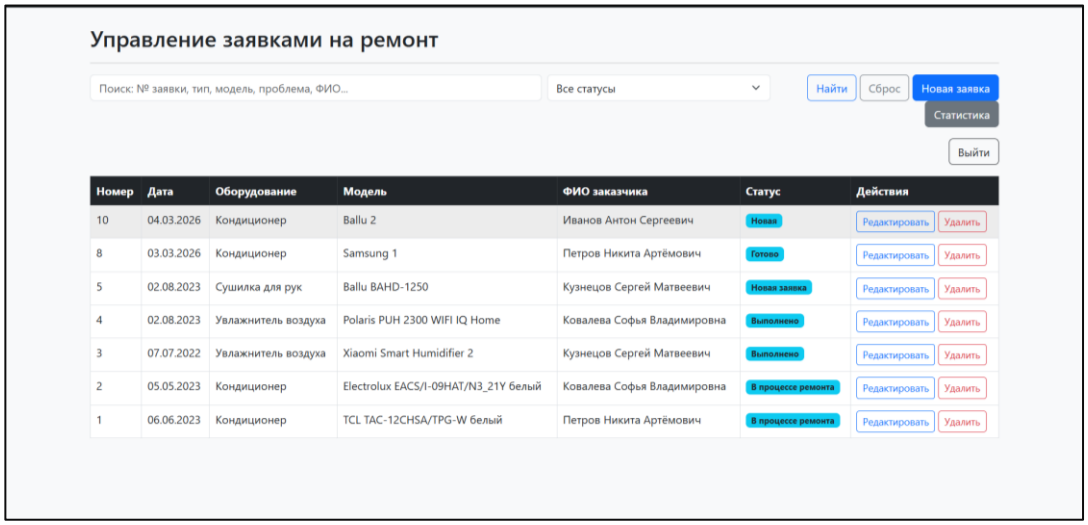

Рисунок 1.8 – Добавленная запись в таблице

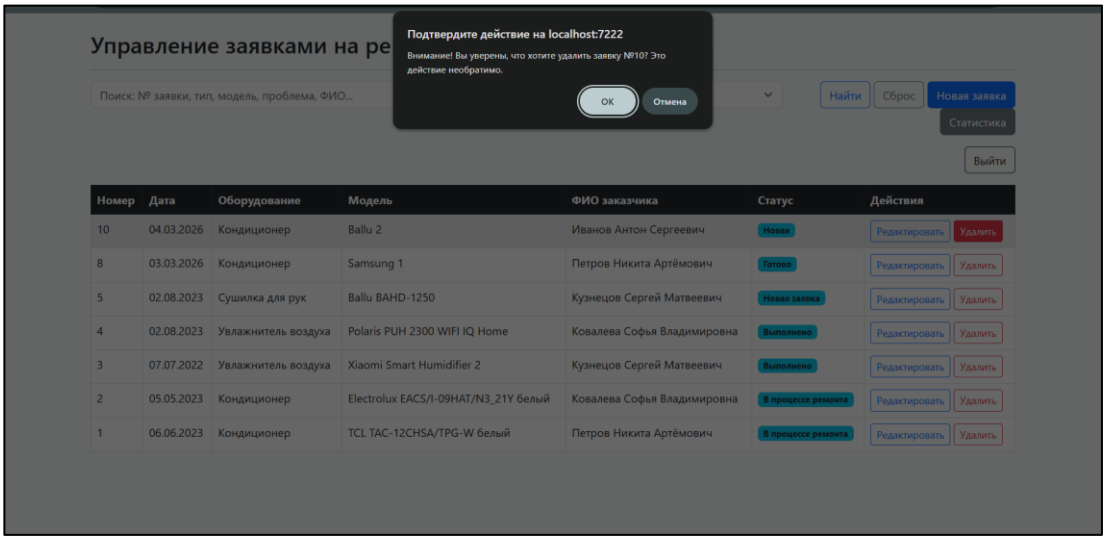

Рисунок 1.9 – Удаление записи в таблице

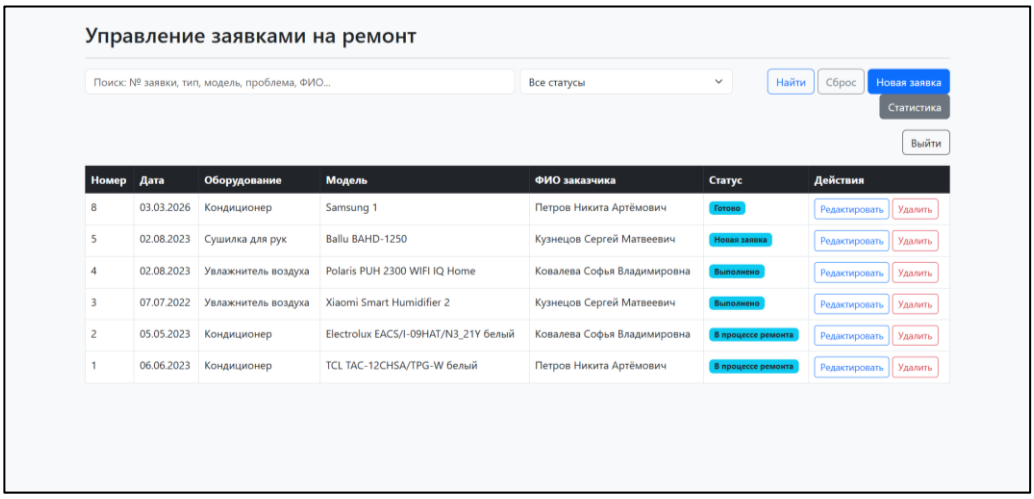

Рисунок 1.10 – Обновленная таблица после удаление записи

1.6 Тестирование программы

После реализации программы она была протестирована, тесты и их результаты представлены в таблице 1.1.

Таблица 1.1 – Тестирование программы

№	Проверяемая функция	Входные данные	Ожидаемый результат	Фактический результат	Статус
1	Добавление заявки	Корректные данные (тип, модель, проблема, ФИО, телефон)	Запись сохранена в БД, добавлена в список	Сохранено, отображается в списке	Успешно
2	Валидация формы	Пустое поле «Описание проблемы»	Блокировка сохранения, вывод ошибки	Сохранение заблокировано	Успешно
3	Редактирование	Изменение статуса на «в процессе ремонта»	Статус обновлен в БД и интерфейсе	Статус успешно обновлен	Успешно
4	Удаление заявки	Нажатие кнопки «Удалить»	Появление запроса на подтверждение	Окно подтверждения отображается	Успешно
5	Расчет статистики	Переход в раздел (при 3 выполненных заявках в БД)	Вывод числа «3» в поле выполненных заявок	Подсчет совпадает с данными БД	Успешно

По результатам тестирование, все тестирование прошли успешно.

1.7 Руководство системному программисту

1.7.1 Общие сведения о программе

Программа «Учёт заявок на ремонт климатического оборудования» предназначена для регистрации, хранения и обработки заявок на ремонт оргтехники и климатического оборудования. Система обеспечивает централизованный учёт заявок, распределение задач между специалистами и контроль сроков выполнения работ.

1.7.2 Функции программы

Основные функции программы:

- регистрация новых заявок (заказчиком и оператором);
- хранение информации о типе и модели оборудования, описании неисправности, заказанных комплектующих;
- назначение ответственного специалиста по заявке;
- изменение статуса заявки (новая, в процессе ремонта, выполнено, отменено и др.);
- ведение комментариев по заявке (специалисты);
- просмотр списка заявок с фильтрацией и поиском по параметрам;
- формирование простой статистики (количество выполненных заявок, среднее время ремонта, распределение по типам неисправностей);
- аутентификация пользователей по логину/паролю и разграничение прав по ролям (заказчик, оператор, специалист, менеджер по качеству).

1.7.3 Минимальный состав технических средств

Минимальная конфигурация для эксплуатации:

- персональный компьютер или сервер с архитектурой x64;
- процессор не ниже 2-ядерного, 2,0 ГГц и выше;
- оперативная память – не менее 4 ГБ (рекомендуется 8 ГБ и более);
- свободное место на диске – не менее 2 ГБ для системы, БД и логов;
- сетевое подключение (для доступа пользователей через браузер);
- монитор с разрешением не ниже 1366×768.

1.7.4 Минимальный состав программных средств

- операционная система: Windows 10/11 (или серверная версия Windows Server с поддержкой .NET 8.0);
- установленный .NET 8.0 Runtime / SDK;
- СУБД PostgreSQL версии 12 и выше;

веб-сервер (по выбору):

- IIS с модулем ASP.NET Core Hosting Bundle, либо
- встроенный Kestrel (запуск через dotnet run);
- браузер пользователя: Microsoft Edge, Google Chrome, Mozilla Firefox (актуальные версии).

1.7.5 Требования к персоналу (системному программисту)

Системный программист, сопровождающий программу, должен:

- знать основы администрирования Windows и PostgreSQL;
- разбираться в работе ASP.NET Core приложений (структура проекта, деплой, логирование);
- уметь работать с SQL (создание/изменение таблиц, резервное копирование, восстановление БД);

разбираться в механизмах аутентификации на базе cookie и настройке прав доступа;

уметь читать и анализировать журналы ошибок (лог-файлы приложения и PostgreSQL)

1.8 Структура программы

1.8.1 Сведения о структуре программы

Программа реализована как веб-приложение ASP.NET Core MVC и состоит из следующих основных слоёв:

- точка входа и конфигурация сервисов (Program.cs);
- слой доступа к данным (Entity Framework Core, AppDbContext и классы моделей);
- контроллеры (логика обработки запросов);
- модели представления (ViewModel) для передачи данных в представления;
- представления (Razor Pages, .cshtml) для отображения интерфейса пользователю;

- статические ресурсы (wwwroot при необходимости).

1.8.2 Сведения о составных частях программы

Основные составные части:

- точка входа: Program.cs Настройка пайплайна: добавление MVC, регистрации AppDbContext, настройка cookie-аутентификации и маршрутизации.
- контекст БД: Models/AppDbContext.cs Описание подключения к PostgreSQL, конфигурация сущностей Datarequest, Datauser, Datacomment, настройка связей и ключей.

Сущности (модели данных):

1) Datarequest – заявки на ремонт;
2) Datauser – пользователи системы (заказчики, операторы, специалисты, менеджеры);
3) Datacomment – комментарии по заявкам.

Контроллеры:

1) AuthController – вход/выход, определение роли по datausers.type, формирование claims и cookie.
2) HomeContoller – операции с заявками: список, поиск, создание, редактирование, удаление, статистика.

ViewModel:

1) RequestIndexViewModel – данные для главной страницы с таблицей заявок и фильтрами;
2) RequestCreateViewModel – данные для формы создания заявки;
3) RequestEditViewModel – данные для формы редактирования (статус, описание, мастер);
4) StatisticsViewModel – агрегированные данные по статистике.

Представления:

1) Views/Auth/Login.cshtml – форма входа;
2) Views/Auth/AccessDenied.cshtml – страница отказа в доступе;
3) Views/Home/Index.cshtml – список заявок, поиск и фильтрация;
4) Views/Home/Create.cshtml – создание заявки;
5) Views/Home/Edit.cshtml – редактирование заявки;
6) Views/Home/Statistics.cshtml – статистика.

1.8.2 Сведения о связях между составными частями программы

Program.cs регистрирует AppDbContext, контроллеры, аутентификацию и маршруты.

AuthController использует AppDbContext и таблицу datausers для аутентификации и назначения ролей.

HomeContoller использует AppDbContext для работы с datarequests, datausers, datacomments.

ViewModel'и наполняются данными в контроллерах и передаются в соответствующие представления.

Представления (.cshtml) используют свойства ViewModel'ей для отображения данных и генерации форм.

1.8.3 Сведения о связях с другими программами

Программа взаимодействует с сервером PostgreSQL как с внешним компонентом (через ADO.NET / Npgsql).

Возможен запуск под управлением IIS (взаимодействие с системой логирования IIS).

При необходимости может быть добавлена интеграция с другими системами (например, внешние API для уведомлений), но в базовой конфигурации таких связей нет.

1.9 Настройка программы

1.9.1 Настройка на состав технических средств

Обеспечить наличие необходимого объёма ОЗУ и свободного места на диске.

При развертывании на сервере – настроить постоянный доступ к PostgreSQL (локальный или по сети).

При работе через IIS – установить и настроить соответствующую роль/функцию, открыть нужные порты.

1.9.2 Настройка на состав программных средств

Установка драйвера.

В контексте данной программы под «драйвером» понимается:

- драйвер доступа к PostgreSQL пакет Npgsql.EntityFrameworkCore.PostgreSQL;
- при использовании внешнего COM-оборудования драйвера COM-порта, однако в текущей конфигурации приложение не использует COM-порты.

Шаги:

Установить PostgreSQL и убедиться, что сервер запущен.

Проверить наличие пакета Npgsql.EntityFrameworkCore.PostgreSQL в проекте (Project_DemoExam.csproj).

При необходимости обновить пакет через dotnet add package или NuGet.

1.10 Проверка программы

1.10.1 Описание способов проверки

Основные способы проверки корректности работы:

1) Функциональное тестирование:

- проверка сценариев входа/выхода и разграничения прав по ролям;
- проверка CRUD-операций по заявкам (создание, просмотр, изменение, удаление);
- проверка поиска и фильтрации заявок;
- проверка формирования статистики.

2) Тестирование интеграции с БД:

- проверка подключения к PostgreSQL;
- проверка корректности работы с первичными/внешними ключами и ограничениями;
- проверка автоинкремента PK и обработки ошибок 23505.

3) Тестирование отказоустойчивости:

- поведение при недоступной БД (корректное сообщение об ошибке, отсутствие падения приложения);
- корректная обработка ошибок сохранения (DbUpdateException, PostgresException).

По результатам испытаний заполняется отдельный документ «Протокол тестирования» с перечнем тест-кейсов, фактическими результатами и статусом.

1.11 Сообщения системному программисту

В процессе работы возможны следующие типичные сообщения и ситуации (таблица 1.2):

Таблица 1.2 – Сообщение, причины и решение

Сообщение / ситуация	Возможная причина	Действия системного программиста
Невозможно подключиться к базе данных	Неверная строка подключения, БД недоступна, служба PostgreSQL остановлена	Проверить строку подключения, доступность порта 5432, запустить службу PostgreSQL
Исключение 23505 по *_pkey	Несогласованность sequence и фактического MAX(id), ручные вставки в БД	Проверить работу встроенного механизма ресида sequence; при необходимости выполнить setval вручную
Ошибка авторизации, сообщение «Неверный логин или пароль»	Неверно введён пароль или логин; данные в datausers некорректны	Проверить запись пользователя в datausers, при необходимости временно установить тестовый пароль
Отказ в доступе (AccessDenied / Forbid) при входе на страницу	Пользователь не имеет нужной роли или обращается к чужим данным	Проверить роли, присвоенные пользователям, логику в [Authorize] и проверках User.IsInRole
Пустой список клиентов в форме создания заявки у оператора	В datausers.type не используются ключевые слова «клиент»/«заказчик»	Уточнить значения поля type в datausers или скорректировать сопоставление типов в коде
Ошибки генерации Razor-представлений	Некорректный синтаксис Razor (@{}, скобки, строки)	Проверить указанные файлы .cshtml, исправить выражения, пересобрать проект

2 Дипломный проект

2.1 Введение дипломного проекта

В современных условиях цифровизации предприятий пищевой промышленности особое значение приобретает автоматизация процессов учета производства. На многих молочных заводах используется программное обеспечение для ведения учета, однако существующие решения часто не соответствуют конкретным требованиям предприятия, имеют ограниченный функционал или неудобный интерфейс.

Актуальность темы заключается в необходимости создания нового программного продукта, который будет учитывать специфику работы предприятия по производству молочной продукции и обеспечит удобный, точный и эффективный учет производственных процессов. Автоматизация позволит повысить достоверность данных, ускорить обработку информации, упростить контроль движения сырья и готовой продукции, а также снизить вероятность ошибок, возникающих при ручном вводе данных.

Цель дипломного проекта – разработать веб-приложение для учета производства молочной продукции на предприятии.

Для достижения поставленной цели необходимо решить следующие задачи:

1) проанализировать предметную область и определить основные процессы учета производства молочной продукции;

2) спроектировать структуру базы данных, обеспечивающую хранение информации о видах продукции, партиях, сырье, сотрудниках и складах;

3) разработать интерфейс пользователя, обеспечивающий удобный ввод, редактирование и просмотр информации;

4) реализовать функционал поиска, фильтрации и сортировки данных по заданным параметрам (дате, виду продукции, партии, складу, сотруднику и т. д.);

5) провести тестирование программы и оценить ее эффективность в решении поставленных задач.

2.2 Анализ предметной области

ООО «Месягутовский молочноконсервный комбинат» находится в Республике Башкортостан и является одним из крупнейших переработчиков молока-сырья в республике. Партнерами предприятия являются более 120 сельхозтоваропроизводителей и фермеров из Башкортостана и соседних регионов. Структура комбината включает несколько подразделений: аппаратный цех, кисломолочный цех, цех сушки молока, маслоцех, лабораторию, инженерно-техническую службу, автопарк, механическую и энергетическую службы.

Предприятие функционирует как сложная производственно-логистическая система: сырье поступает от поставщиков, перерабатывается в различных цехах,

выпускается готовая продукция с определенными сроками годности, хранится на складе и отгружается потребителям (детские сады, больницы, школы и др.). Важен учет множества параметров: поступление и расход сырья, контроль качества, выпуск партий продукции с учетом объемов и сроков годности, хранение на складах, отгрузка и контроль остатков.

При этом существующие процессы часто ведутся в отдельных системах и таблицах, часть данных фиксируется вручную. Это создает ряд проблем: отсутствует единая система, которая интегрировала бы все этапы учета; сложно обеспечить оперативный доступ к информации о движении партий продукции, остатках на складах, сроках годности и ответственных сотрудниках; затруднен поиск, фильтрация и сортировка данных по ключевым параметрам. Кроме того, сохраняется риск ошибок при ручном вводе и дублирования записей, что замедляет формирование отчетности и усложняет планирование производства.

В связи с этим формируется задача: разработать систему для учета производства молочной продукции на предприятии. Она должна обеспечивать ввод, хранение, изменение и удаление данных о сырье, поступлениях, производственных партиях, готовой продукции, складах и сотрудниках, предоставлять удобный интерфейс для поиска и фильтрации информации, а также формировать актуальные отчеты об остатках и производстве. Внедрение подобной системы позволит повысить прозрачность процессов, снизить вероятность ошибок, ускорить доступ к информации и улучшить эффективность управления предприятием.

2.3 Требование к своему программному продукту

Разрабатываемое веб-приложение предназначено для учета производства и складских операций на предприятии, занимающемся выпуском молочной продукции. Система должна обеспечивать удобную работу пользователей, надежное хранение данных и простоту администрирования.

Функциональные требования.

Программный продукт должен обеспечивать выполнение следующих функций:

Учет производимой продукции:

– ведение справочников номенклатуры (материалы, готовая продукция, сырье и т.д.), складов. Для каждой позиции фиксируются характеристики – номер, наименование и так далее в зависимости от справочников;

– фиксация объемов произведенной продукции. При завершении производственного процесса мастер производства заносит данные о количестве произведенной продукции за смену или за конкретную дату;

– учет даты выпуска партий. Каждая партия продукции регистрируется с указанием даты выпуска, что обеспечивает возможность отслеживания сроков хранения и формирования отчетности.

Учет сырья и материалов:

– регистрация поступления сырья на склад. При поступлении новой партии сырья кладовщик вносит данные о наименовании, количестве, поставщике и дате поступления. Система автоматически обновляет складские остатки;

– учет расхода сырья на производство. При запуске производственного цикла система позволяет списывать сырье и материалы, используемое для выпуска продукции. Списание может выполняться автоматически на основании производственных норм;

– контроль остатков на складе. Система в реальном времени отображает актуальные остатки по каждому виду сырья.

Управление складом:

– ведение учета готовой продукции на складе. После завершения производства готовая продукция поступает на склад и регистрируется в системе. Фиксируется количество, дата поступления и место хранения;

– отражение движения товаров (поступление, списание). Предусмотрены операции поступления продукции, а также списания (например, по сроку годности или браку). Каждое движение фиксируется в журнале операций;

– формирование отчетов по остаткам и оборотам. Пользователь может формировать отчеты по текущим остаткам, движению продукции за выбранный период.

Работа с пользователями:

– возможность регистрации и авторизации пользователей. Пользователи системы проходят процедуру регистрации с указанием логина, пароля. Для входа в систему реализована форма авторизации.

Отчетность и аналитика:

– формирование отчетов по производству за выбранный период. Система формирует отчеты по объему произведенной продукции за произвольный период. В отчетах указываются склад, даты, виды продукции и суммарные показатели;

– экспорт отчетов в формате PDF/Excel. Все отчеты могут быть выгружены в форматах PDF и Excel для дальнейшего хранения, печати или отправки руководству.

Нефункциональные требования.

1) Удобство интерфейса:

– интерфейс должен быть интуитивно понятным и адаптированным для пользователей без специальной подготовки;

– реализована навигация по основным разделам через меню.

2) Надежность и безопасность:

– доступ к данным осуществляется только после авторизации;

– защита от несанкционированного изменения информации;

– хранение данных в СУБД postgresql с возможностью резервного копирования.

3) Производительность

– поддержка одновременной работы нескольких пользователей.

4) Портативность и совместимость

– приложение должно корректно работать в современных браузерах (Google Chrome, Mozilla Firefox, Microsoft Edge);

– возможность развертывания на локальном сервере или в облачной среде.

5) Расширяемость

– возможность добавления новых модулей (например, учет логистики или поставщиков) без серьезной переработки основной архитектуры.

Эксплуатационные требования.

Для корректной и стабильной работы системы необходимо соблюдение определенных эксплуатационных условий, связанных с характеристиками серверного и клиентского оборудования, а также с качеством интернет-соединения. Ниже приведены системные требования, обеспечивающие надежную эксплуатацию веб-приложения.

Системные требования:

1) Серверная часть

Минимальные требования:

– версии ОС: Windows 10;

– оперативная память: 4 ГБ;

– процессор: 2-ядерный;

– свободное место на диске: 15 ГБ;

Рекомендованные требования:

– версии ОС: Windows 11;

– оперативная память: 8 ГБ и выше;

– процессор: 4–6 ядер;

– свободное место на диске: 20–50 ГБ;

2) Клиентская часть

Минимальные требования:

– веб-браузер: Google Chrome, Mozilla Firefox, Microsoft Edge (актуальные версии);

– процессор: 2-ядерный;

– операционная система: Windows 10;

– свободное место на диске: 5 ГБ;

– оперативная память: 2 ГБ.

Рекомендованные требования:

– процессор: 4-ядерный;

– оперативная память: 4 ГБ и выше;

– свободное место на диске: 5 ГБ;

– операционная система: Windows 11.

2.4 Обоснование выбора инструментальных средств разработки

Для реализации веб-приложения по учету производства молочной продукции были выбраны следующие инструментальные средства: язык программирования C#, фреймворк ASP.NET, система управления базами данных PostgreSQL, а также технологии HTML, CSS и JavaScript для создания пользовательского интерфейса.

Для разработки веб-приложения выбран язык C# и платформа ASP.NET. C# – современный объектно-ориентированный язык с строгой типизацией, что помогает делать программы более надежными и снижает количество ошибок. ASP.NET позволяет удобно создавать динамические веб-приложения, работать с базами данных и обеспечивать безопасность. Кроме того, хорошая интеграция с Visual Studio и .NET Framework делает процесс разработки и тестирования проще.

Рассматривались и другие варианты: Python, PHP и Java. Python легко учить и использовать для быстрого прототипирования, но он медленнее C# и менее строгий, что может создавать проблемы в больших проектах. PHP тоже популярный язык для веб-разработки, но структура кода не всегда строгая, а вопросы безопасности требуют внимания. Java с фреймворком Spring подходит для крупных корпоративных решений, но требует больше ресурсов и сложнее настраивается.

C# обеспечивает хорошую производительность и стабильность, а также позволяет поддерживать код чистым и структурированным. Этот язык лучше подходит для проектов, где важны надежность, масштабируемость и долгосрочная поддержка. Python может быть удобен для небольших или быстрых проектов, где скорость разработки важнее производительности.

Поэтому для этого проекта выбор C# и ASP.NET выглядит оптимальным: приложение будет работать быстро, надежно, а поддерживать его будет удобно.

Для хранения данных о производстве выбрана система PostgreSQL. Эта база данных считается надежной и производительной, а еще она бесплатная и с открытым исходным кодом. PostgreSQL поддерживает стандарт SQL, транзакции, хранимые процедуры и триггеры, что удобно для реализации даже сложной логики работы приложения.

Рассматривались и другие варианты, например MySQL и Microsoft SQL Server. MySQL проще в установке и настройке, но ее возможности для сложных запросов ограничены. SQL Server хорошо работает с C#, но это коммерческий продукт, что увеличивает затраты.

PostgreSQL оказалась оптимальным выбором для этого проекта: она бесплатная, стабильная, работает на разных платформах и отлично интегрируется с .NET.

Клиентская часть: HTML, CSS, JavaScript и Figma.

Для создания пользовательского интерфейса используются HTML, CSS и JavaScript. HTML задает структуру страниц, CSS отвечает за оформление и

адаптацию интерфейса под разные устройства, а JavaScript делает страницы интерактивными и позволяет обновлять данные без полной перезагрузки.

Можно использовать современные фреймворки, такие как React или Vue.js, но их настройка усложняет проект. Использование базовых технологий позволяет реализовать все необходимые функции без лишней сложности.

Перед началом разработки интерфейса планируется использовать Figma для проектирования макетов и прототипов страниц. В Figma можно визуально продумать расположение элементов, цветовую гамму и удобство навигации, что помогает создать удобный и понятный интерфейс до начала программирования.

Вывод: выбор стека C#, ASP.NET, PostgreSQL, HTML, CSS, JavaScript, Figma обеспечивает высокую производительность, надежность и удобство работы с приложением. Эти технологии позволяют создавать стабильное, масштабируемое и удобное веб-приложение с современным интерфейсом и безопасной работой с данными.

2.5 Разработка архитектуры блок-схем программы

В данном разделе представлена блок-схема, отражающая общий алгоритм функционирования разработанного веб-приложения, предназначенного для учета производства молочной продукции на предприятии. Использование блок-схемы позволяет визуально представить последовательность операций и взаимодействие основных модулей системы, что упрощает понимание логики работы приложения и служит основой для его дальнейшей программной реализации.

При проектировании структуры веб-приложения была использована модульная архитектура, включающая механизмы авторизации пользователей, учета поступления сырья, регистрации производственных операций, управления номенклатурой, а также формирования итоговой отчетности. Блок-схема отражает взаимодействие этих компонентов в рамках единого алгоритма обработки данных. В ней последовательно представлены этапы получения входной информации, ее валидации, выполнения соответствующих действий и формирования результата, возвращаемого пользователю.

На рисунке 2.1 приведена разработанная архитектура блок-схемы веб-приложения.

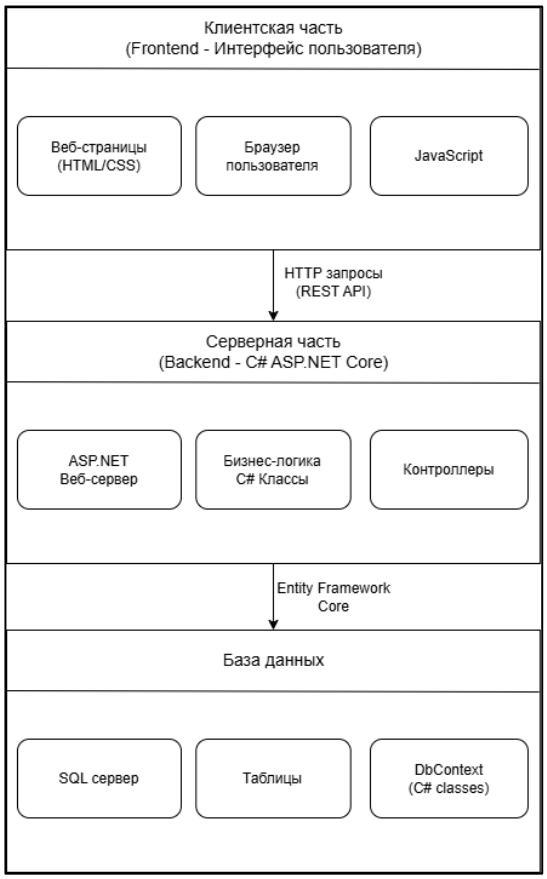

Рисунок 2.1 – Схема архитектуры веб-приложения

2.6 Разработка макетов основных форм приложения

На этапе проектирования веб-приложения важной задачей является разработка макетов пользовательских форм, которые определяют внешний вид интерфейса и структуру взаимодействия пользователя с системой. Макеты служат прототипами будущих страниц и позволяют заранее продумать логику размещения элементов управления, механизмы ввода данных и способы отображения информации. Такой подход позволяет упростить последующую реализацию интерфейса, снизить вероятность ошибок и обеспечить удобство работы операторов.

При создании макетов учитывались особенности предметной области – необходимость быстрого ввода производственных данных, точность фиксации показателей и минимизация количества действий при выполнении операций. Поэтому при разработке форм использовались принципы простоты, последовательности и интуитивной понятности. Все формы были разделены на несколько функциональных групп в соответствии с задачами системы.

Основные макеты включают:

1) форму авторизации пользователя, предназначенную для ввода логина и пароля. Макет содержит минимальный набор элементов: поля ввода, кнопку входа

и ссылку для восстановления доступа. Структура формы выполнена так, чтобы обеспечить быстрый доступ к системе (Рисунок 2.2).

Рисунок 2.2 – Макет авторизации

2) главную страницу (панель навигации), обеспечивающую переход ко всем ключевым разделам приложения. В макете предусмотрено боковое меню, содержащее пункты. Такой подход обеспечивает удобную навигацию и упрощает ориентацию пользователя в интерфейсе (Рисунок 2.3).

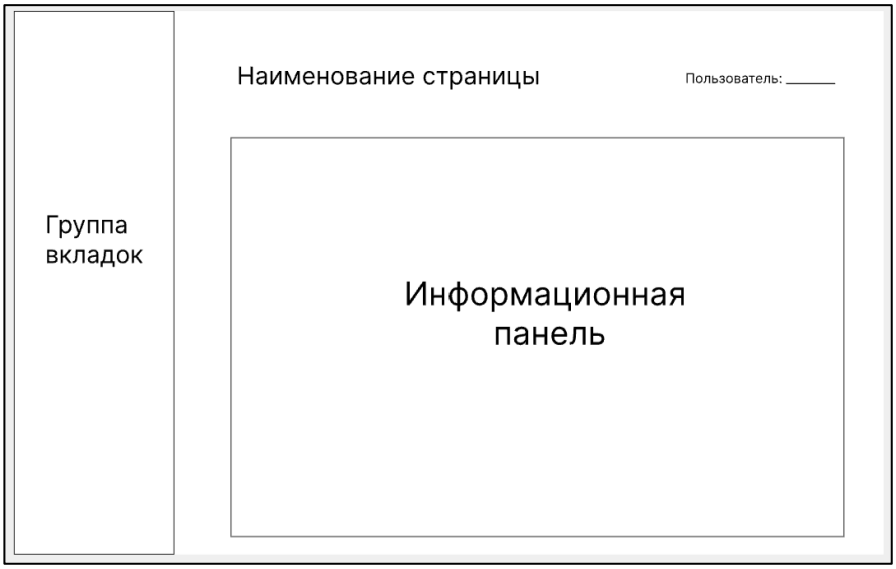

Рисунок 2.3 – Макет главной страницы

3) форму учета поступления сырья, включающую таблицу с обязательными параметрами: дата, поставщик, количество сырого молока, показатели качества и

назначение сырья. Макет предусматривает возможность добавления, редактирования и сохранения записей в базе данных (Рисунок 2.4).

Рисунок 2.4 – Макет страницы для учета сырья

4) форму регистрации производственных операций, предназначенную для фиксации факта переработки сырья, выпуска готовой молочной продукции и списания материалов. Макет содержит поля для выбора номенклатуры, указания количества, даты и связанных технологических параметров (Рисунок 2.5).

Рисунок 2.5 – Макет страницы регистрации операций

Каждый макет проектировался с учетом принципов адаптивности, что позволяет корректно отображать страницы как на настольных компьютерах, так и на мобильных устройствах. На основе разработанных макетов была выполнена

последующая реализация интерфейса веб-приложения с использованием современных веб-технологий.

2.7 Проектирование и разработка базы данных

Проектирование базы данных является одним из ключевых этапов разработки информационной системы учёта молочной продукции на предприятии. Целью данного этапа является создание структуры хранения данных, которая обеспечивает корректное отражение всех процессов: поступление сырья, учёт готовой продукции, перемещение товаров между складами, списание и формирование отчётности.

На основе анализа предметной области были определены основные сущности и взаимосвязи, характерные для деятельности предприятия. В качестве базовых элементов были выделены: номенклатура товаров, склады, контрагенты, подразделения, документы поступления, позиции документов, а также сведения об остатках на складах. Эти сущности образуют фундаментальную логику системы и обеспечивают полное отражение движения материальных ценностей.

Структура базы данных включает следующие основные таблицы:

1) nomenclatures – номенклатура;
– id: serial primary key;
– group_name: varchar(200);
– name: varchar(200);
– base_unit: varchar(50);
– type: varchar(200);
– comment: text;
– number: int.

2) warehouses – таблица складов с указанием их наименования и местоположения;
– id: serial primary key;
– name: varchar(200);
– type: varchar(200);
– address: text;
– number: int.

3) departments – структурные подразделения предприятия;
– id: serial primary key;
– name: varchar(200);
– parent_id: int foreign key.

4) counterparties – информация о контрагентах;
– id: serial primary key;
– name: varchar(200);
– inn: varchar(12);
– contact: varchar(255).

5) receipts – таблица документов поступления, содержащая номер документа, дату, ответственного, склад получения и другие реквизиты;

– id: serial primary key;
– doc_number: varchar(50);
– doc_ref: varchar(50);
– date: date;
– counterparty_id: int foreign key;
– warehouse_id: int foreign key;
– responsible: varchar(100);
– comment: text.

6) receiptitems – позиции товаров в документе поступления, связанные с конкретным документом и номенклатурой;

– id: serial primary key;
– receipt_id: int foreign key;
– nomenclature_id: int foreign key;
– quantity: numeric;
– price: numeric.

7) warehouse_movements – текущие остатки товаров на складах, обновляемые при операциях поступления, списания и перемещения;

– id: serial primary key;
– warehouse_id: int foreign key;
– nomenclature_id: int foreign key;
– initial_balance: numeric;
– incoming: numeric;
– outgoing: numeric;
– final_balance: numeric;
– comment: text.
– date: date.

8) production_materials – материалы, списанные на производство в рамках отчёта: номенклатура, склад, количество;

– id: serial primary key;
– production_report_id: int foreign key;
– nomenclature_id: int foreign key;
– quantity: numeric;

9) production_products – готовая продукция, произведённая по отчёту: номенклатура, количество, выход продукции;

– id: serial primary key;
– production_report_id: int foreign key;
– nomenclature_id: int foreign key;
– quantity: numeric;
– batch_number: integer;
– shelf_life: integer;

10) productionreport – документ отчёта производства, фиксирующий результат переработки сырья в готовую продукции за определённый период;

– id: serial primary key;
– doc_number: varchar(50);
– date: date;
– department_id: int foreign key;
– warehouse_id: int foreign key;
– responsible: varchar(100);
– comment: text.

11) writeoffitems – позиции списания, включающие конкретную номенклатуру, количество и склад;

– id: serial primary key;
– writeoff_id: int foreign key;
– nomenclature_id: int foreign key;
– quantity: numeric.
– created_at: timestap.

12) writeoffs – документы списания, используемые при порче, истечении срока;

– id: serial primary key;
– doc_number: varchar(50);
– date: date;
– warehouse_id: int foreign key;
– operation_type: varchar(50);
– responsible: varchar(100);
– reason: varchar(100);
– comment: text;
– basis: text;
– created_at: timestap.

13) writeoffitems –позиции списания, включающие конкретную номенклатуру, количество и склад;

– id: serial primary key;
– writeoff_id: int foreign key;
– nomenclature_id: int foreign key;
– quantity: numeric.

14) users – учетные записи пользователей;

– id: serial primary key;
– username: varchar(50);
– password: varchar(50);
– full_name: varchar(100);
– role: varchar(20);

15) units – единица измерения номенклатур.

– id: serial primary key;
– name: varchar(50).

Между таблицами установлены связи через первичные и внешние ключи. Для номенклатуры, складов и документов используются связи один-ко-многим, что обеспечивает логическую целостность данных. Все таблицы приведены к

нормальным формам, что позволяет избежать избыточности, предотвратить дублирование информации и повысить стабильность работы системы. Внешние ключи обеспечивают защиту от некорректного удаления или изменения связанных данных.

При физической реализации модели в СУБД PostgreSQL были выбраны оптимальные типы данных (integer, varchar, numeric, date, text). На уровне базы данных реализованы механизмы автоматического обновления остатков с помощью триггеров, что обеспечивает корректность данных при любом изменении связанных документов. Структура БД была разработана таким образом, чтобы допускать дальнейшее расширение и включение дополнительных модулей учёта, не нарушая существующую архитектуру.

Ниже на рисунке 2.6 представлена ERD-диаграмма, отражающая структуру базы данных и основные взаимосвязи между сущностями.

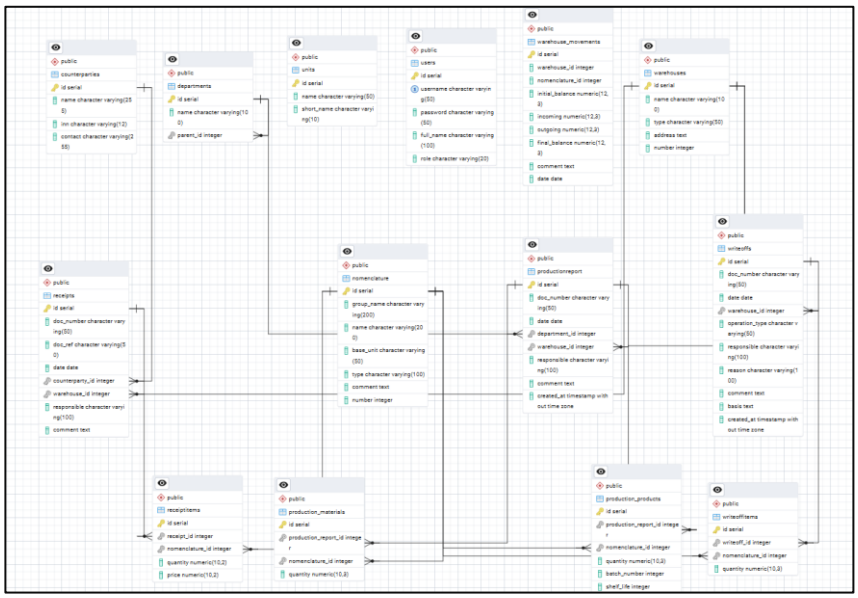

Рисунок 2.6 – ERD-диаграмма БД

2.8 Разработка пользовательского интерфейса

Страница «Номенклатура». Пользовательский интерфейс веб-приложения состоит из двух основных областей: боковой панели навигации и основной рабочей области. Боковая панель содержит меню выбора разделов системы. Основная область включает заголовок, панель управления с кнопками и таблицу данных (Рисунок 2.7).

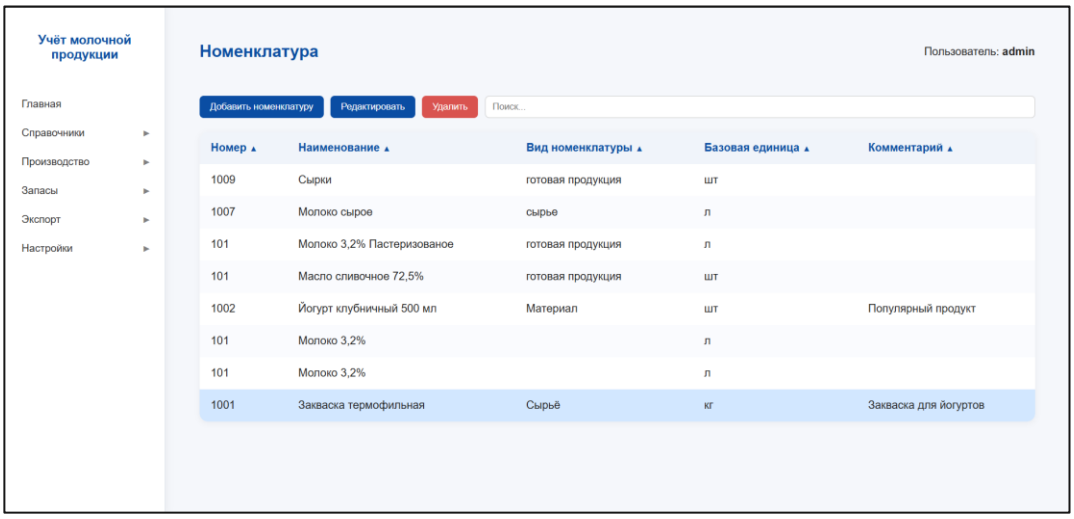

Рисунок 2.7 – Общий вид интерфейса приложения

Для удобства работы с данными реализован полнотекстовый поиск по таблице и сортировка по всем колонкам. Поиск выполняется в реальном времени по мере ввода текста пользователем. Функции поиска и сортировки представлены в листинге 2.1.

Листинг 2.1 – Функции поиска и сортировки

```
        // Поиск по таблице
        searchInput.addEventListener("input", () => {
          const filter = searchInput.value.toLowerCase();
          tableBody.querySelectorAll("tr").forEach(row => {
            const match = Array.from(row.children).some(td =>
              td.innerText.toLowerCase().includes(filter));
            row.style.display = match ? "" : "none";
          });
        });

        // Сортировка по клику на заголовок
        headers.forEach((th, index) => {
          th.addEventListener("click", () => {
            const rowsArray =
Array.from(tableBody.querySelectorAll("tr"));
            const ascending = !th.asc;
            rowsArray.sort((a, b) => {
              const aText =
a.children[index].innerText.toLowerCase();
              const bText =
b.children[index].innerText.toLowerCase();
              return ascending ? aText.localeCompare(bText) :
bText.localeCompare(aText);
            });
            rowsArray.forEach(row => tableBody.appendChild(row));
          });
        });
```

Для добавления и редактирования записей используется модальное окно с формой ввода данных. Внешний вид модального окна добавления записи показан на рисунке 2.8.

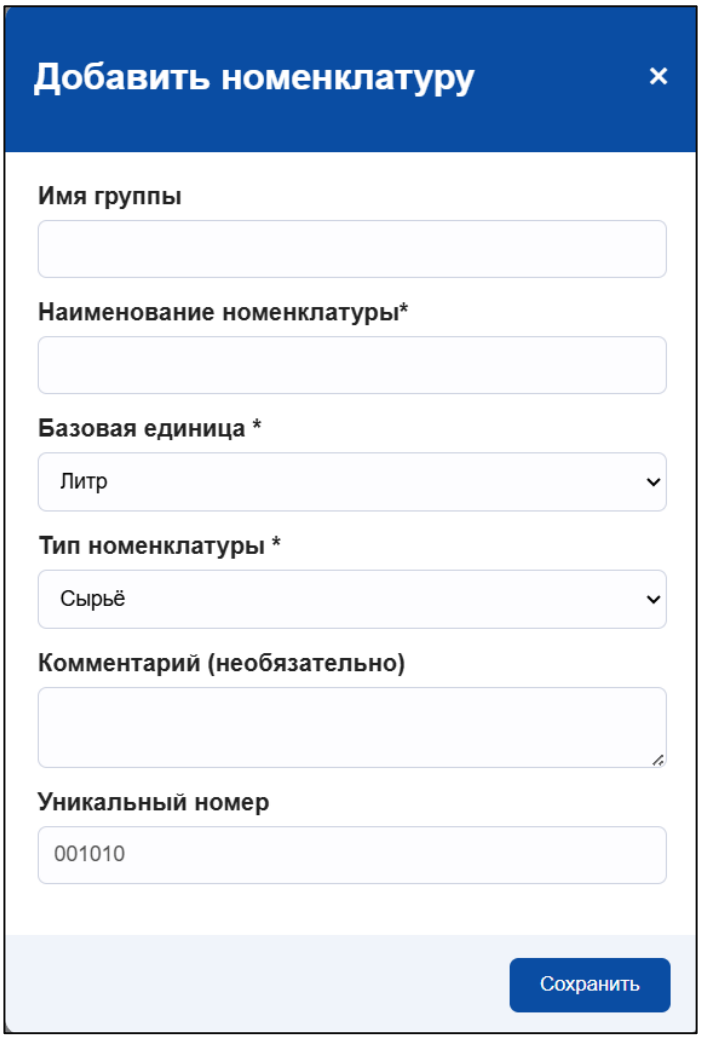

Рисунок 2.8 – Модальное окно добавления записи

При нажатии на кнопку «Добавить» открывается форма с автоматически сгенерированным номером позиции.

Для удаления записей реализовано подтверждающее диалоговое окно. Пользователь должен предварительно выбрать строку в таблице, после чего может удалить её через кнопку «Удалить». Код функции удаления записи представлен в листинге 2.2.

Листинг 2.2 – Функция удаления записи

```
deleteBtn.addEventListener("click", async () => {
  const row = tableBody.querySelector("tr.selected");
  if (!row) {
    alert("Выберите строку для удаления");
    return;
  }
```

```
        if (!confirm("Удалить запись?")) return;

        const id = row.dataset.id;
        try {
          await fetch(`/api/nomenclatures/${id}`, { method:
"DELETE" });
          row.remove();
        } catch (e) {
          alert("Ошибка удаления");
        }
      });
```

Интерфейс обеспечивает полный цикл работы с данными номенклатуры: просмотр, добавление, редактирование, удаление, поиск и сортировку. Все операции выполняются без перезагрузки страницы через асинхронные запросы к API, что обеспечивает удобство и эффективность работы с системой.

Страница «Создание поступления».

Интерфейс страницы создания поступления товаров состоит из боковой панели навигации и основной рабочей области. Боковая панель содержит многоуровневое меню с разделами системы, а основная область включает форму ввода данных документа и таблицу для добавления товарных позиций. Общий вид интерфейса представлен на рисунке 2.9.

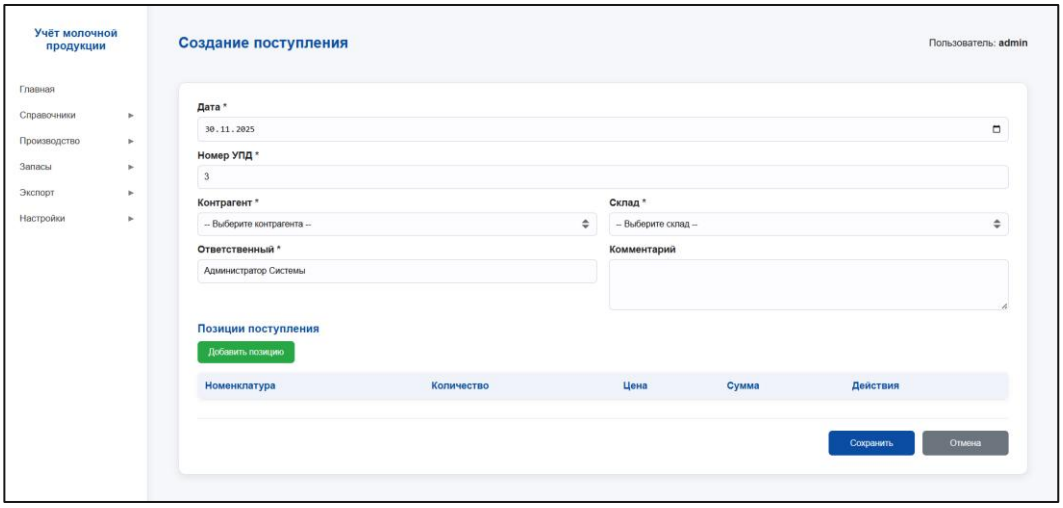

Рисунок 2.9 – Общий вид страницы создания поступления

HTML-структура страницы организована с использованием семантических тегов. Основная форма содержит поля для ввода реквизитов документа: даты, номера УПД, контрагента, склада и ответственного лица (Листинг 2.3).

Листинг 2.3 – HTML-структура формы создания поступления

```
      <div class="form-container">
          <div class="form-row">
              <div class="form-group">
                  <label>Дата *</label>
                  <input type="date" id="receiptDate"
required>
              </div>
              <div class="form-group">
                  <label>Номер УПД *</label>
                  <input type="text" id="receiptNumber"
required>
              </div>
          </div>

          <div class="positions-section">
              <h3>Позиции поступления</h3>
              <button id="addPositionBtn">Добавить
позицию</button>
              <table>
                  <thead>
                      <tr>
                          <th>Номенклатура</th>
                          <th>Количество</th>
                          <th>Цена</th>
                          <th>Сумма</th>
                          <th>Действия</th>
                      </tr>
                  </thead>
                  <tbody id="TableBody"></tbody>
              </table>
          </div>
      </div>
```

Для визуального оформления элементов интерфейса применялись каскадные таблицы стилей. Форма имеет двухколоночную структуру с адаптивным расположением полей. Таблица позиций включает столбцы для выбора номенклатуры, ввода количества, цены и автоматического расчета суммы. Основные стилевые правила представлены в листинге 2.4.

Листинг 2.4 – CSS-стили формы и таблицы позиций

```
      .form-row {
          display: flex;
          gap: 20px;
          margin-bottom: 14px;
      }

      .form-row .form-group {
          flex: 1;
```

```
        display: flex;
        flex-direction: column;
    }

    .positions-section table {
        width: 100%;
        border-collapse: collapse;
        border-radius: 8px;
        overflow: hidden;
        box-shadow: 0 2px 8px rgba(0,0,0,0.05);
        background: white;
    }

    .positions-section th,
    .positions-section td {
        padding: 12px;

        text-align: left;
        border-bottom: 1px solid #e0e4ec;
    }
```

Функциональность интерфейса реализована на языке JavaScript. При загрузке страницы выполняются асинхронные запросы к API для получения списков складов и контрагентов. В листинге 2.5 приведен код инициализации формы.

Листинг 2.5 – Функции загрузки справочников

```
    async function loadWarehouses() {
        const select =
document.querySelector("#warehouseSelect");
        select.innerHTML = `<option value="">-- Выберите
склад --</option>`;

        try {
            const response = await fetch("/api/warehouses");
            if (!response.ok) throw new Error(`Ошибка
сервера: ${response.status}`);
            const data = await response.json();

            data.forEach(w => {
                const option =
document.createElement("option");
                option.value = w.id;
                option.textContent = w.name;
                select.appendChild(option);
            });
        } catch (err) {
            console.error("Ошибка при загрузке складов:",
err);
```

```
                alert("Не удалось загрузить склады");
            }
        }
```

Для работы с товарными позициями реализована динамическая система добавления и удаления строк в таблице. При нажатии кнопки «Добавить позицию» создается новая строка с выпадающим списком номенклатуры и полями для ввода количественных характеристик. В листинге 2.6 показан логика работы с позициями.

Листинг 2.6 – Функции управления позициями

```
        let receiptItems = [];

        async function addReceiptItem() {
            const newItem = {
                id: null,
                nomenclature_id: null,
                nomenclature_name: "",
                quantity: 0,
                price: 0,
                sum: 0
            };
            receiptItems.push(newItem);
            const row = document.createElement("tr");

            row.innerHTML = `
                <td>
                    <select class="nomenclatureSelect">
                        <option value="">-- Выберите
номенклатуру --</option>
                        ${nomenclatureOptions.map(n =>
                            `<option
value="${n.id}">${n.name}</option>`
                        ).join("")}
                    </select>
                </td>
                <td><input type="number" class="quantityInput"
value="0" min="0"></td>
                <td><input type="number" class="priceInput"
value="0" min="0" step="0.01"></td>
                <td class="sumCell">0</td>
                <td><button
class="deleteBtn">Удалить</button></td>
            `;

            tableBody.appendChild(row);
        }
```

Расчет суммы позиции выполняется автоматически при изменении значений количества или цены. При вводе данных в соответствующие поля происходит пересчет и отображение итоговой суммы в реальном времени. Алгоритм расчета представлен в листинге 2.7.

Листинг 2.7 – Функция расчета суммы позиции

```
     function updateRowSum() {
         newItem.sum = newItem.quantity *
newItem.price;
         row.querySelector(".sumCell").textContent =
newItem.sum.toFixed(2);
     }
     // Обработчики событий для полей ввода

     qtyInput.addEventListener("input", () => {
         newItem.quantity =
parseFloat(qtyInput.value) || 0;
         updateRowSum();
     });

     priceInput.addEventListener("input", () => {
         newItem.price = parseFloat(priceInput.value)
|| 0;
         updateRowSum();
     });
```

Валидация данных формы осуществляется при сохранении документа. Проверяются обязательные поля заголовка документа и наличие хотя бы одной товарной позиции. При обнаружении ошибок пользователь получает уведомления. Код валидации и сохранения приведен в листинге 2.8.

Листинг 2.8 – Функция сохранения документа поступления

```
     async function saveReceipt() {
         // Проверка обязательных полей
         if (!receiptDate || !receiptNumber ||
isNaN(counterpartyId) ||
             isNaN(warehouseId) || !responsible) {
             alert("Заполните все обязательные поля!");
             return;
         }

         // Проверка наличия позиций
         const rows = tableBody.querySelectorAll("tr");
         if (rows.length === 0) {
             alert("Добавьте хотя бы одну позицию!");
```

```
            return;
        }
        // Формирование данных для отправки
        const receiptData = {
            doc_number: receiptNumber,
            date: receiptDate,
            counterparty_id: counterpartyId,
            warehouse_id: warehouseId,
            responsible: responsible,
            comment: comment,
            items: items
        };
        // Отправка на сервер
        const response = await fetch("/api/receipts", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify(receiptData)
        });
    }
```

Интерфейс обеспечивает удобное взаимодействие с пользователем через интуитивно понятные элементы управления, мгновенную обратную связь и защиту от некорректного ввода данных. Все операции выполняются без перезагрузки страницы, что повышает удобство работы с системой.

2.9 Доработка приложения

2.9.1 Разработка блок-схем функций

Блок-схема функций построения архитектурных моделей.

Для начала необходимо создать блок-схему функции перемещения продукции, чтобы детально визуализировать логику движения товарно-материальных ценностей (Рисунок 2.10).

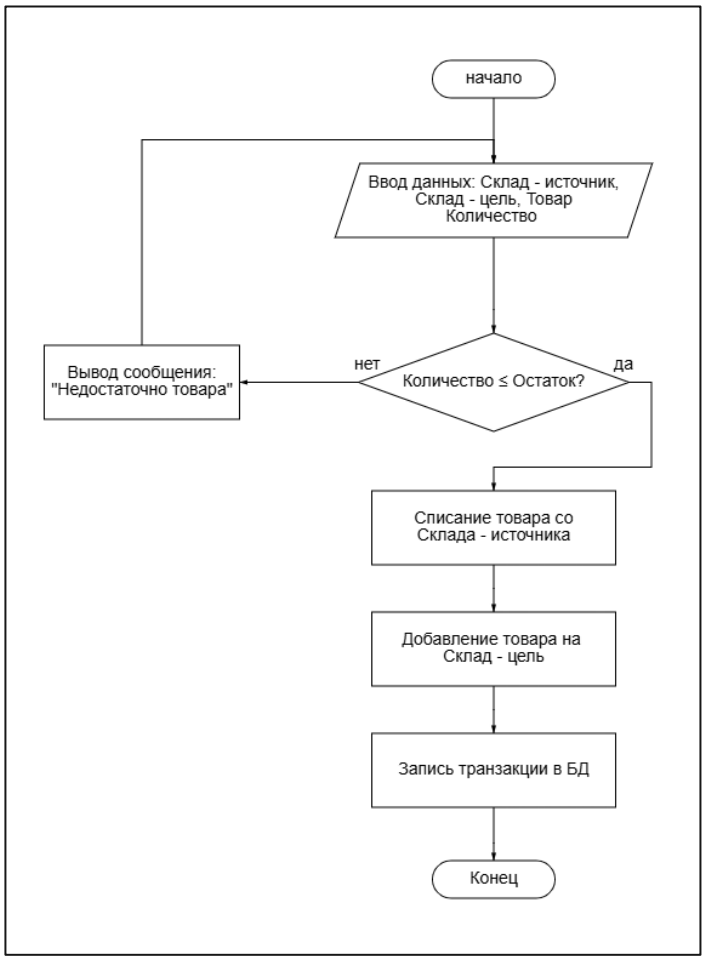

Рисунок 2.10 – Блок-схема перемещения товаров

Данная блок-схема описывает алгоритм перемещения молочной продукции между складами предприятия. Работа начинается с инициализации формы перемещения, где пользователь выбирает исходный склад (откуда списывается товар) и целевой склад (куда товар поступает). Далее происходит выбор конкретной номенклатуры продукции и ввод необходимого количества.

Ключевым этапом является автоматическая проверка остатков: система сверяет введенное количество с фактическим наличием товара на исходном складе в базе данных. Если товара недостаточно, выводится сообщение об ошибке, и процесс возвращается к этапу ввода количества. Если проверка пройдена успешно, транзакция подтверждается, происходит обновление остатков в базе данных для обоих складов, и операция завершается. Такой подход исключает возможность возникновения отрицательных остатков и пересортицы.

Блок-схема функции смены темы оформления.

Следующая блок-схема описывает процесс работы функции переключения цветовой темы интерфейса, обеспечивающей эргономичность работы пользователя (Рисунок 2.11).

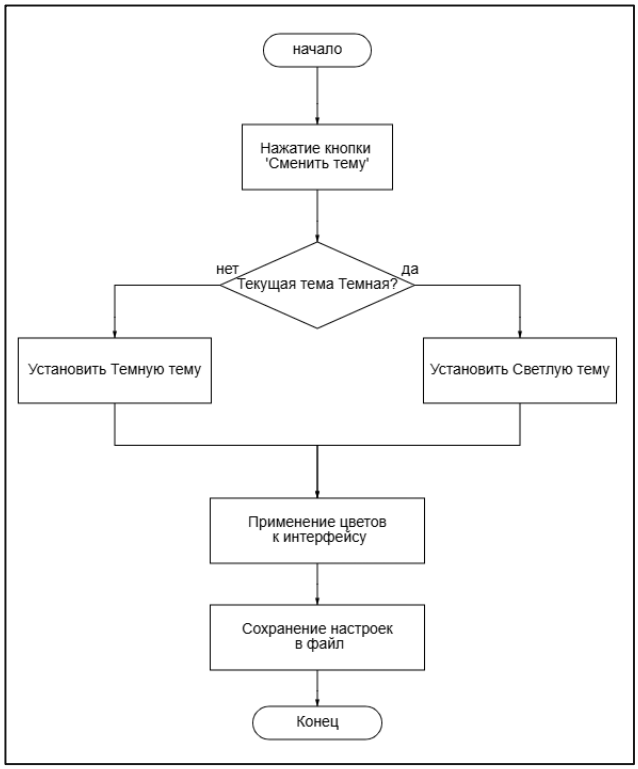

Рисунок 2.11 – Блок-схема смены темы оформления

Разработка блок-схемы переключения темы начинается с действия пользователя (нажатие на кнопку переключения). После этого система считывает текущее состояние конфигурации (активна «светлая» или «темная» тема). На основе полученных данных алгоритм производит инверсию цветовой палитры: если была светлая – применяется темная, и наоборот.

Важным этапом является сохранение настроек: новая конфигурация записывается в локальный файл настроек пользователя. Это необходимо для того, чтобы при следующем запуске программа открывалась в выбранном пользователем режиме. Процесс завершается обновлением всех элементов управления (кнопок, таблиц, фона) в соответствии с новой палитрой. Такая логика обеспечивает персонализацию рабочего места и повышает удобство использования программы.

Блок-схема функции быстрого поиска

Следующая блок-схема описывает процесс работы модуля глобального поиска, предназначенного для оперативной навигации и нахождения информации в системе (Рисунок 2.12).

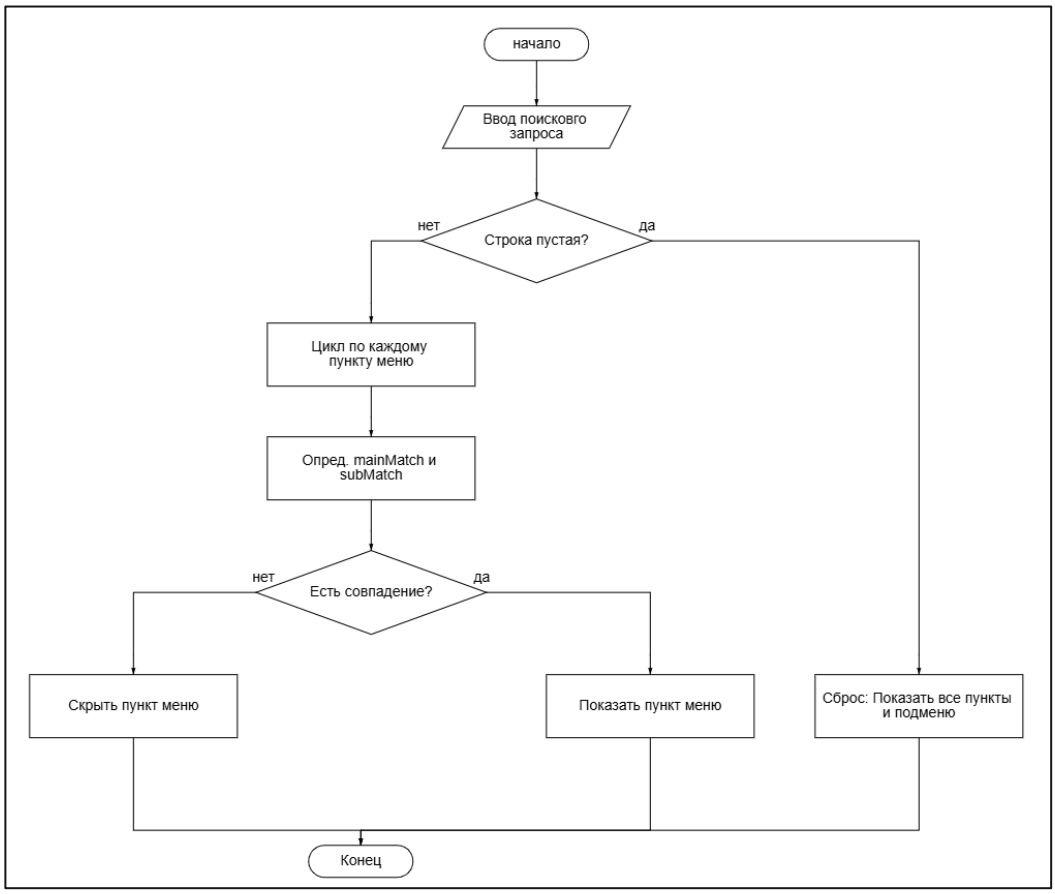

Рисунок 2.12 – Блок-схемы быстрого поиска

Процесс начинается с ввода пользователем поисковой строки. На первом этапе система проверяет, является ли строка пустой. Если поле очищено (пустая строка), алгоритм сбрасывает все фильтры, отображая все пункты и подменю в исходном виде, после чего завершает работу.

Если строка содержит символы, запускается цикл проверки для каждого пункта навигации. Система определяет два параметра: mainMatch (совпадение в заголовке раздела) и subMatch (совпадение в подпунктах).

2.9.2 Разработка функционала

Разработка функции перемещения товаров

Для реализации модуля учета внутренних перемещений товарно-материальных ценностей была разработана веб-страница, использующая технологии HTML5 для разметки и JavaScript для обработки событий. Данный интерфейс предназначен для просмотра реестра перемещений и управления документами движения продукции между складами.

В верхней части рабочей области реализована панель управления показана на рисунке 3.1, содержащая функциональные кнопки:

«Добавить» – осуществляет переход на форму создания нового документа перемещения;

«Редактировать» – позволяет внести изменения в выбранный документ;

«Удалить» – предназначена для удаления ошибочно созданных.

Рядом с кнопками управления расположено поле динамического поиска (input type="text"), позволяющее фильтровать записи в таблице, по ключевым словам, в реальном времени.

Центральным элементом интерфейса является интерактивная таблица (transfersTable), отображающая журнал операций. В таблице реализованы следующие столбцы с функцией сортировки данных (при клике на заголовок):

- номер документа;
- дата операции;
- склад-отправитель («Откуда»);
- склад-получатель («Куда»);
- количество перемещаемой продукции;
- ответственное лицо.

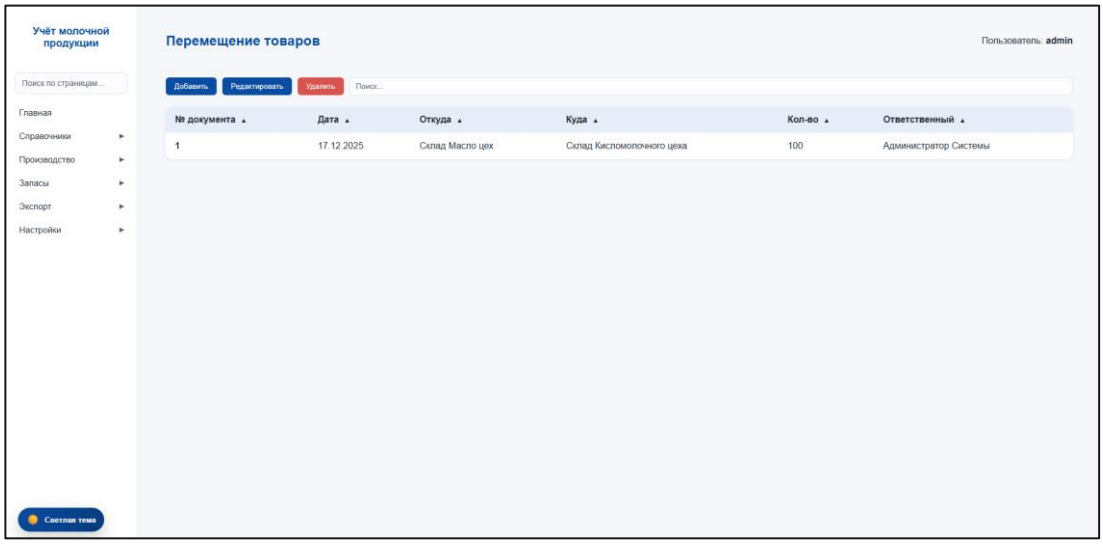

Рисунок 2.13– Интерфейс реестра перемещения товаров

При переходе к созданию новой записи пользователю открывается форма, включающая выпадающие списки для выбора склада-отправителя и склада-получателя. Формирование товарной части документа производится в динамической таблице, куда пользователь добавляет необходимые номенклатурные позиции.

На уровне клиентских скриптов реализована строгая валидация вводимых данных: при попытке указать количество продукции, превышающее текущий фактический остаток на складе-отправителе, система блокирует подтверждение формы и выводит уведомление об ошибке. Функция автоматического контроля остатков обеспечивает логическую целостность данных еще до отправки их в базу, предотвращая возникновение пересортицы.

Завершение операции осуществляется управляющей кнопкой сохранения, которая инициирует запись транзакции. После этого документ отображается в общем журнале перемещений, где фиксируется история операций с указанием даты, маршрута движения товара и ответственного сотрудника. Визуальное оформление формы выполнено в едином стиле с основным приложением

Разработка функции смены темы оформления

Функция смены цветовой темы интерфейса реализована для улучшения эргономики работы и снижения нагрузки на зрение пользователей (Рисунок 2.14).

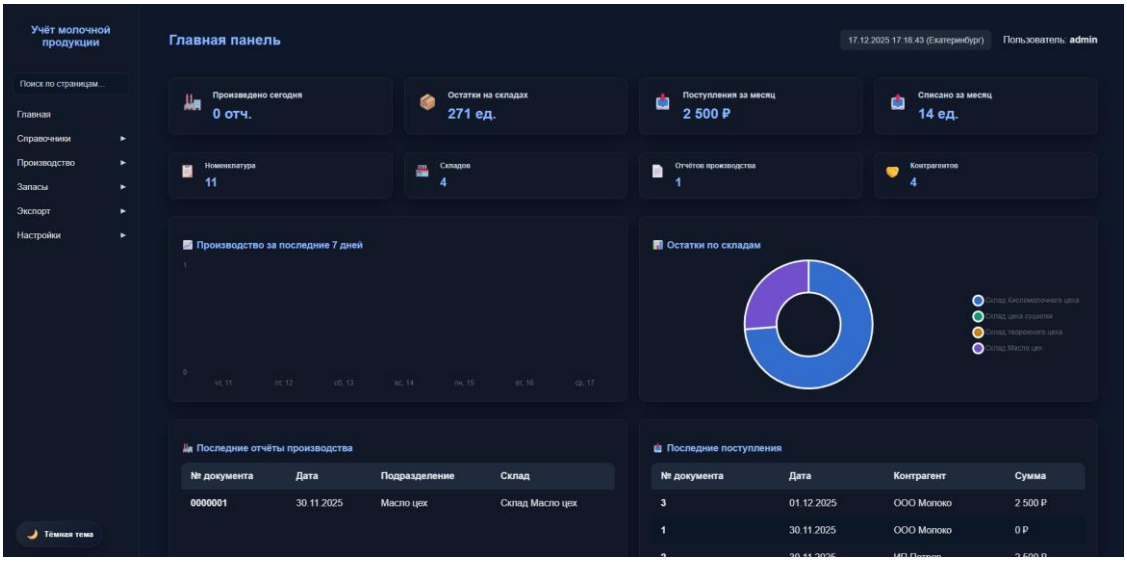

Рисунок 2.14 – Переключение в режим темной темы

Функция управления темой предоставляет пользователю возможность адаптировать интерфейс под условия освещенности. Техническая реализация базируется на использовании CSS-переменных для цветов фона, текста и элементов управления. При переключении темы скрипт меняет атрибут у корневого элемента документа, что мгновенно обновляет палитру всего приложения без перезагрузки страницы.

Важной частью разработки стало сохранение настроек пользователя. При выборе темы значение записывается в локальное хранилище браузера. При следующем запуске приложения скрипт инициализации проверяет сохраненное значение и автоматически применяет нужную тему, либо ориентируется на системные настройки операционной системы. Такой подход обеспечивает персонализацию рабочего места и повышает комфорт длительной работы с табличными данными.

Разработка функции быстрого поиска по системе

Разработка функции глобального поиска предназначена для оперативной навигации по разделам системы и справочникам (Рисунок 2.15).

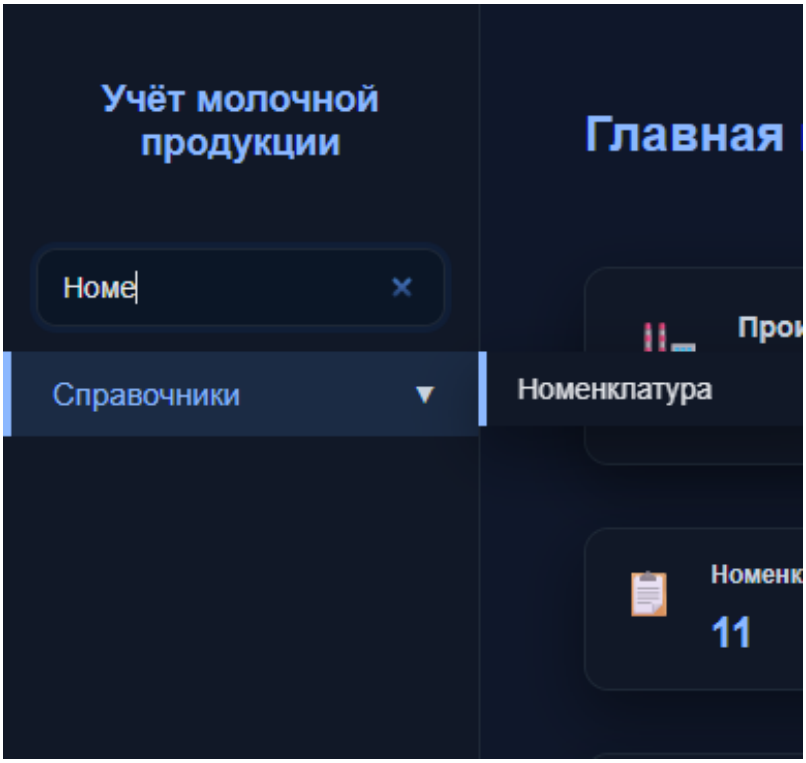

Рисунок 2.15 – Работа функции быстрого поиска

Функция поиска реализована на языке JavaScript и обеспечивает фильтрацию навигационного меню в реальном времени. Алгоритм работы начинается с отслеживания ввода символов пользователем. Если поисковая строка пуста, система сбрасывает фильтры, отображая все пункты и подменю в исходном состоянии. При вводе запроса запускается цикл проверки, который анализирует как заголовки основных разделов, так и названия вложенных пунктов.

Логика отображения построена следующим образом: если совпадение найдено в заголовке раздела, он раскрывается полностью, предоставляя доступ ко всем вложенным функциям. Если совпадение найдено только в подпункте, родительский раздел остается видимым, но внутри него скрываются нерелевантные элементы. Это позволяет пользователю мгновенно находить нужные отчеты или справочники (например, «Номенклатура» или «Списание»), не переходя вручную по всем вкладкам меню. Данное решение значительно ускоряет работу оператора и повышает общую эффективность взаимодействия с системой.

Заключение

В процессе прохождения учебной практики все поставленные цели и задачи были успешно достигнуты. Основным результатом проделанной работы стала полностью готовая программа и пакет технической документации для сдачи демонстрационного экзамена, что также послужило надежной практической базой для реализации модуля дипломного проекта.

Для достижения этого результата был пройден полный цикл разработки информационной системы:

- спроектирована структура базы данных и написан функциональный программный код;
- настроена система безопасности с разграничением прав доступа пользователей по ролям;
- разработан удобный, интуитивно понятный и защищенный пользовательский интерфейс;
- составлена и оформлена вся необходимая проектная документация.

Все этапы создания проекта последовательно фиксировались в системе контроля версий Git. Итоговый программный продукт функционирует стабильно, без сбоев и эффективно решает реальные бизнес-задачи организации.

Библиография

1 Документация по платформе ASP.NET Core и языку C# [Электронный ресурс] – Режим доступа: https://learn.microsoft.com/ru-ru/aspnet/core/;

2 Официальная документация СУБД PostgreSQL [Электронный ресурс] – Режим доступа: https://www.postgresql.org/docs/;

3 Руководство по Entity Framework Core для работы с базами данных [Электронный ресурс] – Режим доступа: https://learn.microsoft.com/ru-ru/ef/core/;

4 Интегрированная среда разработки Visual Studio: официальное руководство [Электронный ресурс] – Режим доступа: https://learn.microsoft.com/ru-ru/visualstudio/;

5 MDN Web Docs: Веб-технологии для разработчиков (HTML, CSS, JavaScript) [Электронный ресурс] – Режим доступа: https://developer.mozilla.org/ru/;

6 Руководство пользователя инструмента администрирования баз данных pgAdmin 4 [Электронный ресурс] – Режим доступа: https://www.pgadmin.org/docs/;

7 Официальная документация системы контроля версий Git [Электронный ресурс] – Режим доступа: https://git-scm.com/doc;

8 ГОСТ 19.701-90 ЕСПД. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения [Электронный ресурс] – Режим доступа: https://docs.cntd.ru/document/1200007206.

Приложение А
(справочная)
Листинг программы

Листинг создание базы данных

```
-- 1. Создание таблицы Пользователей
CREATE TABLE datausers (
    userID SERIAL PRIMARY KEY,
    fio VARCHAR(150) NOT NULL,
    phone VARCHAR(20),       -- Тип VARCHAR, чтобы
большие номера телефонов помещались без ошибок
    login VARCHAR(50) NOT NULL,
    password VARCHAR(50) NOT NULL,
    type VARCHAR(50) NOT NULL
);

-- 2. Создание таблицы Заявок
CREATE TABLE datarequests (
    requestID SERIAL PRIMARY KEY,
    startDate DATE NOT NULL,
    orgTechType VARCHAR(100) NOT NULL,
    orgTechModel VARCHAR(100) NOT NULL,
    problemDesc TEXT NOT NULL,
    requestStatus VARCHAR(50) NOT NULL,
    completionDate DATE,
    repairParts TEXT,
    masterID INT,
    clientID INT NOT NULL,
    -- Связи (внешние ключи) с таблицей пользователей
    FOREIGN KEY (masterID) REFERENCES datausers(userID),
    FOREIGN KEY (clientID) REFERENCES datausers(userID)
);

-- 3. Создание таблицы Комментариев
CREATE TABLE datacomments (
    commentID SERIAL PRIMARY KEY,
    message TEXT NOT NULL,
    masterID INT NOT NULL,
    requestID INT NOT NULL,
    -- Связи (внешние ключи) с пользователями и заявками
    FOREIGN KEY (masterID) REFERENCES datausers(userID),
    FOREIGN KEY (requestID) REFERENCES
datarequests(requestID)
);
```

Приложение Б
(справочная)
Листинг программы

Файл Program.cs

```
using Microsoft.EntityFrameworkCore;
using Microsoft.AspNetCore.Authentication.Cookies;
using Project_DemoExam.Models;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();
builder.Services.AddDbContext<AppDbContext>();

builder.Services

.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme
)
    .AddCookie(options =>
    {
        options.LoginPath = "/Auth/Login";
        options.AccessDeniedPath = "/Auth/AccessDenied";
        options.SlidingExpiration = true;
    });

builder.Services.AddAuthorization();

var app = builder.Build();

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

Файл AppDbContext.cs

```
using System;
using System.Collections.Generic;
using Microsoft.EntityFrameworkCore;

namespace Project_DemoExam.Models;

public partial class AppDbContext : DbContext
{
    public AppDbContext()
    {
    }

    public AppDbContext(DbContextOptions<AppDbContext>
options)
        : base(options)
    {
    }

    public virtual DbSet<Datacomment> Datacomments { get; set;
}

    public virtual DbSet<Datarequest> Datarequests { get; set;
}

    public virtual DbSet<Datauser> Datausers { get; set; }

    protected override void
OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    #warning To protect potentially sensitive information in your
connection string, you should move it out of source code. You can
avoid scaffolding the connection string by using the Name= syntax to
read it from configuration - see
https://go.microsoft.com/fwlink/?linkid=2131148. For more guidance
on storing connection strings, see
https://go.microsoft.com/fwlink/?LinkId=723263.
        =>
optionsBuilder.UseNpgsql("Host=localhost;Port=5432;Database=Demoexam
s;Username=postgres;Password=admin");

    protected override void OnModelCreating(ModelBuilder
modelBuilder)
    {
        modelBuilder.Entity<Datacomment>(entity =>
        {
            entity.HasKey(e =>
e.Commentid).HasName("datacomments_pkey");

            entity.ToTable("datacomments");
```

```
                    entity.Property(e =>
e.Commentid).ValueGeneratedOnAdd().HasColumnName("commentid");
                    entity.Property(e =>
e.Masterid).HasColumnName("masterid");
                    entity.Property(e =>
e.Message).HasColumnName("message");
                    entity.Property(e =>
e.Requestid).HasColumnName("requestid");

                    entity.HasOne(d => d.Master).WithMany(p =>
p.Datacomments)
                        .HasForeignKey(d => d.Masterid)
                        .OnDelete(DeleteBehavior.ClientSetNull)

.HasConstraintName("datacomments_masterid_fkey");

                    entity.HasOne(d => d.Request).WithMany(p =>
p.Datacomments)
                        .HasForeignKey(d => d.Requestid)
                        .OnDelete(DeleteBehavior.ClientSetNull)

.HasConstraintName("datacomments_requestid_fkey");
                });

                modelBuilder.Entity<Datarequest>(entity =>
                {
                    entity.HasKey(e =>
e.Requestid).HasName("datarequests_pkey");

                    entity.ToTable("datarequests");

                    entity.Property(e =>
e.Requestid).ValueGeneratedOnAdd().HasColumnName("requestid");
                    entity.Property(e =>
e.Clientid).HasColumnName("clientid");
                    entity.Property(e =>
e.Completiondate).HasColumnName("completiondate");
                    entity.Property(e =>
e.Masterid).HasColumnName("masterid");
                    entity.Property(e => e.Orgtechmodel)
                        .HasMaxLength(100)
                        .HasColumnName("orgtechmodel");
                    entity.Property(e => e.Orgtechtype)
                        .HasMaxLength(100)
                        .HasColumnName("orgtechtype");
                    entity.Property(e =>
e.Problemdesc).HasColumnName("problemdesc");
                    entity.Property(e =>
e.Repairparts).HasColumnName("repairparts");
                    entity.Property(e => e.Requeststatus)
                        .HasMaxLength(50)
                        .HasColumnName("requeststatus");
```

```
                    entity.Property(e =>
e.Startdate).HasColumnName("startdate");

                    entity.HasOne(d => d.Client).WithMany(p =>
p.DatarequestClients)
                        .HasForeignKey(d => d.Clientid)
                        .OnDelete(DeleteBehavior.ClientSetNull)

.HasConstraintName("datarequests_clientid_fkey");

                    entity.HasOne(d => d.Master).WithMany(p =>
p.DatarequestMasters)
                        .HasForeignKey(d => d.Masterid)

.HasConstraintName("datarequests_masterid_fkey");
                });

                modelBuilder.Entity<Datauser>(entity =>
                {
                    entity.HasKey(e =>
e.Userid).HasName("datausers_pkey");

                    entity.ToTable("datausers");

                    entity.Property(e =>
e.Userid).ValueGeneratedOnAdd().HasColumnName("userid");
                    entity.Property(e => e.Fio)
                        .HasMaxLength(150)
                        .HasColumnName("fio");
                    entity.Property(e => e.Login)
                        .HasMaxLength(50)
                        .HasColumnName("login");
                    entity.Property(e => e.Password)
                        .HasMaxLength(50)
                        .HasColumnName("password");
                    entity.Property(e => e.Phone)
                        .HasMaxLength(20)
                        .HasColumnName("phone");
                    entity.Property(e => e.Type)
                        .HasMaxLength(50)
                        .HasColumnName("type");
                });

                OnModelCreatingPartial(modelBuilder);
            }

            partial void OnModelCreatingPartial(ModelBuilder
modelBuilder);
        }
```